

The Deterministic Agent — Build Plan

Target: Complete CORE + STRETCH in ~2 days

Stack: Python 3.12, Docker Compose (Python service + Redis + Postgres), FastAPI, Pydantic v2

LLM Strategy: OpenRouter (single API key, all models) with per-phase model routing:
Planning → Gemini 2.5 Flash (\$0.30/\$2.50/1M) · Coding → Claude Sonnet 4 (\$3/\$15/1M)
Evaluation → GPT-4o-mini (\$0.15/\$0.60/1M) · Recovery → Claude Sonnet 4 (\$3/\$15/1M)

Estimated cost: ~\$0.10/run, ~\$0.50 for all 5 recorded runs

Eval weights to keep in mind: Problem Decomposition (25%), Architecture & Extensibility (25%), Systems Thinking (20%), Code Quality & Tests (15%), Stretch & Real-Time (10%), README (5%)

Build Order & Time Budget

Phase	What	Time	Cumulative	Goals Covered
0	Project scaffold + Docker + infra	1.5h	1.5h	Architecture, Systems Thinking
1	Core data models + interfaces	1.5h	3h	Problem Decomposition, Architecture
2	LLM provider abstraction + sandbox	2h	5h	Architecture, Extensibility
3	Agent core (plan→code→execute→evaluate)	3h	8h	CORE: The Agent
4	Decision tracing	2h	10h	CORE: Decision Trace
5	Path locking	2.5h	12.5h	CORE: Path Locking
6	CLI	1.5h	14h	CORE: CLI
7	Record 5 runs + tests	2h	16h	CORE: 5 runs, Tests
8	Fork & explore	2h	18h	STRETCH: Fork & Explore
9	Path analysis + visualization	1.5h	19.5h	STRETCH: Path Analysis
10	FastAPI endpoints	2h	21.5h	STRETCH: API
11	Real-time streaming	2h	23.5h	STRETCH: Streaming
12	README + final polish	1.5h	25h	README & Reasoning

Phase 0: Project Scaffold + Docker + Infrastructure

What we're building

The complete project skeleton with Docker Compose orchestrating three services: the Python application, Redis (for caching + pub/sub streaming), and Postgres (for persistent trace storage). All configuration via `.env` files. This phase produces zero business logic but establishes the patterns every subsequent phase inherits.

Which evaluation criteria this addresses

- **Architecture & Extensibility (25%):** Clean project structure, dependency injection groundwork, config management
- **Systems Thinking (20%):** Containerized from day one, health checks, persistent volumes, graceful degradation if Redis is down

Project structure to create

```
deterministic-agent/
├─ docker-compose.yml
├─ Dockerfile
├─ .env                # Local dev defaults
├─ .env.example        # Checked into git (no secrets)
├─ .dockerignore
├─ .gitignore
├─ pyproject.toml      # Dependencies, tool config
├─ alembic.ini         # DB migrations config
├─ alembic/
│   └─ env.py
│   └─ versions/
├─ src/
│   └─ __init__.py
│   └─ config.py        # Pydantic Settings, reads .env
│   └─ dependencies.py  # DI container / factory
│   └─ models/          # Pydantic domain models
│       └─ __init__.py
│   └─ db/              # SQLAlchemy models + session
│       └─ __init__.py
│       └─ base.py
│       └─ session.py
│       └─ models.py
│   └─ cache/           # Redis + in-memory cache
│       └─ __init__.py
│   └─ agent/           # Agent core
│       └─ __init__.py
```

```

|   ├── llm/                                # LLM provider abstraction
|   |   └── __init__.py
|   ├── sandbox/                            # Code execution
|   |   └── __init__.py
|   ├── tracing/                            # Decision trace capture
|   |   └── __init__.py
|   ├── locking/                            # Path locking
|   |   └── __init__.py
|   ├── forking/                            # Fork & explore
|   |   └── __init__.py
|   ├── analysis/                           # Path analysis
|   |   └── __init__.py
|   ├── api/                                # FastAPI routes
|   |   └── __init__.py
|   ├── cli/                                # CLI commands
|   |   └── __init__.py
|   └── streaming/                           # Real-time events
|       └── __init__.py
└── tests/
    ├── conftest.py
    ├── unit/
    │   └── integration/
└── scripts/
    └── seed_runs.py                        # Helper to record 5 runs
└── runs/                                  # Default run artifact directory (gitignored)

```

Prompt for AI IDE

I'm building a Python project called "deterministic-agent" – a coding agent system with decision tracing and path locking. The tech stack is:

- Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2.0 (async), Alembic
- Docker Compose with 3 services: app (Python), redis (Redis 7), db (Postgres 16)
- Click for CLI
- pytest + pytest-asyncio for testing

Create the complete project scaffold with these requirements:

1. ****docker-compose.yml****:

- `app` service: builds from Dockerfile, mounts src as volume for hot reload, depends on redis + db, exposes port 8000, reads from .env
- `redis` service: Redis 7, port 6379, no persistence needed for dev
- `db` service: Postgres 16, port 5432, persistent volume, health check with pg_isready
- A shared network for all services

2. ****Dockerfile****:
 - Multi-stage build. Python 3.12-slim base.
 - Install system deps (gcc for some pip packages).
 - Copy pyproject.toml first for layer caching, install deps.
 - Copy src. Don't run as root – create a non-root user.
 - CMD should run uvicorn for the FastAPI app.
3. ****env**** and ****env.example****:

—— LLM Configuration (OpenRouter — single key, all models) ——

We use OpenRouter as a unified gateway to access Claude, GPT, Gemini,

and DeepSeek models with one API key and one bill. The API is

OpenAI-SDK compatible — just a different base_url.

```
LLM_PROVIDER=openrouter LLM_API_KEY=sk-or-v1-your-openrouter-key-here  
LLM_BASE_URL=https://openrouter.ai/api/v1 LLM_TEMPERATURE=0.0  
LLM_MAX_TOKENS=4096
```

—— Per-phase model routing ——

Each agent phase uses the optimal model for its task:

- Planning: Gemini 2.5 Flash — best structured reasoning at \$0.30/\$2.50/1M

(native JSON schema enforcement, built-in thinking mode, 1M context)

- Coding: Claude Sonnet 4 — highest first-pass code success rate at \$3/\$15/1M

(fewer retry cycles saves more tokens than the unit cost premium)

- Evaluation: GPT-4o-mini — cheapest reliable JSON classifier at \$0.15/\$0.60/1M

(this is a pass/fail classification task, not a reasoning task)

- Recovery: Claude Sonnet 4 — code comprehension for debugging at \$3/\$15/1M

(misdiagnosed errors burn 2+ retry cycles, quality pays for itself here)

LLM_MODEL_PLANNING=google/gemini-2.5-flash
LLM_MODEL_CODING=anthropic/claude-sonnet-4
LLM_MODEL_EVALUATION=openai/gpt-4o-mini
LLM_MODEL_RECOVERY=anthropic/claude-sonnet-4
LLM_MODEL_FALLBACK=deepseek/deepseek-chat-v3.2

—— Single-model override (optional) ——

Set this to force ALL phases to use one model (useful for testing or when you want a uniform trace). Overrides all per-phase settings above.

LLM_MODEL_OVERRIDE=anthropic/claude-sonnet-4

Database

DATABASE_URL=postgresql+asyncpg://agent:agent@db:5432/deterministic_agent

Redis

REDIS_URL=redis://redis:6379/0

Agent

AGENT_MAX_ITERATIONS=10
AGENT_EXECUTION_TIMEOUT=120
AGENT_MAX_COST_USD=2.00

Paths

RUNS_DIR=./runs

API

API_HOST=0.0.0.0
API_PORT=8000

4. ****src/config.py****:

Use ``pydantic_settings.BaseSettings`` with a nested model structure:

- ``LLMConfig``:
 - ``provider``: str (default "openrouter")
 - ``api_key``: SecretStr
 - ``base_url``: str (default "https://openrouter.ai/api/v1")
 - ``temperature``: float (default 0.0)
 - ``max_tokens``: int (default 4096)
 - ``model_planning``: str (default "google/gemini-2.5-flash")
 - ``model_coding``: str (default "anthropic/claude-sonnet-4")
 - ``model_evaluation``: str (default "openai/gpt-4o-mini")
 - ``model_recovery``: str (default "anthropic/claude-sonnet-4")
 - ``model_fallback``: str (default "deepseek/deepseek-chat-v3.2")
 - ``model_override``: Optional[str] (default None – if set, overrides all per-phase models)
- ``get_model_for_phase(phase: AgentPhase) -> str`` method that checks override first,
 - then returns the phase-specific model
- ``DatabaseConfig`` (url, echo, pool_size)
- ``RedisConfig`` (url, ttl_default, max_memory_mb)
- ``AgentConfig`` (max_iterations, execution_timeout, max_cost_usd)
- ``Settings`` that composes all of these
- Settings should load from .env file with env_prefix where appropriate
- Include a ``get_settings()`` function with lru_cache for singleton behavior

5. ****pyproject.toml****:

- Use ``[project]`` metadata format (PEP 621)
- Dependencies: fastapi, uvicorn[standard], pydantic>=2.0, pydantic-settings, sqlalchemy[asyncio], asyncpg, alembic, redis[hiredis], click, rich, httpx, openai, yfinance, pandas, numpy, scikit-learn, matplotlib, pytest, pytest-asyncio, pytest-cov
- NOTE: We only need the ``openai`` SDK – OpenRouter is OpenAI-API compatible. No need for ``anthropic`` or ``google-generativeai`` packages. All models (Claude, Gemini, GPT, DeepSeek) are accessed through the same OpenAI SDK with a different base_url. This is a deliberate architecture choice that keeps dependencies minimal.
- Define a CLI entry point: ``deterministic-agent = "src.cli.main:cli"``
- Configure pytest, ruff, mypy in `[tool.*]` sections

6. ****src/db/****:

- ``base.py``: SQLAlchemy declarative base with common columns (id UUID, created_at, updated_at)
- ``session.py``: async engine + sessionmaker factory, reads DATABASE_URL from config
- ``models.py``: empty for now, will be populated in Phase 1

7. ****src/cache/__init__.py****:

- Create the protocol/interface only:

```
```python
```

```
 class CacheBackend(Protocol):
```

```
 async def get(self, key: str) -> Optional[str]: ...
```

```
 async def set(self, key: str, value: str, ttl: Optional[int] = None) ->
```

```
None: ...
```

```
 async def delete(self, key: str) -> None: ...
```

```
 async def exists(self, key: str) -> bool: ...
```

```
```
```

8. ****src/dependencies.py****:

- A simple DI container class that lazily initializes and holds references to: Settings, DB session factory, Redis client, Cache backend
- async ``startup()`` and ``shutdown()`` lifecycle methods
- This is NOT a framework – just a plain class that creates and holds deps

9. ****tests/conftest.py****:

- Fixtures for: test settings (with test DB URL), async db session, redis client mock
- Use ``pytest.mark.asyncio`` as default

10. ****Alembic setup****:

- alembic.ini pointing to src/db for models
- env.py configured for async SQLAlchemy

Follow these coding standards throughout ALL code:

- Type hints on ALL function signatures and return types
- Docstrings on all public classes and functions (Google style)
- Abstract base classes / Protocols for all interfaces
- No bare except clauses – always catch specific exceptions
- Use ``logging`` module with structured logging (not print)
- Constants in UPPER_SNAKE_CASE
- Private methods prefixed with underscore
- Use ``pathlib.Path`` not string paths
- Use ``datetime.datetime`` with timezone awareness (UTC)
- Imports sorted: stdlib, third-party, local (enforced by ruff)

Manual verification checklist

After this phase, verify:

```
bash
```

```
# 1. Docker Compose starts all 3 services
docker-compose up -d
docker-compose ps # All 3 should be "Up" / "healthy"

# 2. Postgres is accessible
docker-compose exec db psql -U agent -d deterministic_agent -c "SELECT 1"

# 3. Redis is accessible
docker-compose exec redis redis-cli ping # Should return PONG

# 4. Python app starts (will have minimal FastAPI app)
curl http://localhost:8000/health # Should return {"status": "ok"}

# 5. Settings load correctly
docker-compose exec app python -c "from src.config import get_settings; print(get_se

# 6. Alembic works
docker-compose exec app alembic upgrade head
docker-compose exec app alembic current # Should show head revision

# 7. Tests run
docker-compose exec app pytest tests/ -v # Should find conftest, pass with 0 tests
```

Phase 1: Core Data Models + Interfaces

What we're building

All Pydantic domain models (decision points, traces, run metadata, cost estimates) and all Protocol/ABC interfaces (LLM provider, cache backend, sandbox, tracer, locker). Zero implementation — just the contracts. This is where Problem Decomposition score lives.

Which evaluation criteria this addresses

- **Problem Decomposition (25%):** The model hierarchy IS the decomposition. Reviewers will look at these models first.
- **Architecture & Extensibility (25%):** Protocols/ABCs mean any engineer can add a new LLM provider or cache backend without touching core logic.

Prompt for AI IDE

```
We're building the core data models and interfaces for the deterministic agent
system.
This phase is MODELS AND INTERFACES ONLY — no implementations yet.
```


The project scaffold already exists at the structure I'll describe. Read `src/config.py` and `src/db/base.py` to understand existing patterns.

Create the following files:

1. `src/models/enums.py`

Define all enums used across the system:

```
```python
class RunStatus(str, Enum):
 PENDING = "pending"
 RUNNING = "running"
 SUCCESS = "success"
 FAILED = "failed"
 PARTIAL = "partial" # Crashed mid-run, trace saved

class AgentPhase(str, Enum):
 PLANNING = "planning"
 CODING = "coding"
 EXECUTING = "executing"
 EVALUATING = "evaluating"
 RECOVERING = "recovering"

class DecisionCategory(str, Enum):
 DATA_SELECTION = "data_selection"
 ALGORITHM_SELECTION = "algorithm_selection"
 ARCHITECTURE = "architecture"
 LIBRARY_SELECTION = "library_selection"
 ERROR_RECOVERY = "error_recovery"
 PARAMETER_TUNING = "parameter_tuning"

class VarianceTier(str, Enum):
 HIGH = "high" # 7.5-10: fundamentally changes output
 MEDIUM = "medium" # 5-7: changes presentation
 LOW = "low" # 2-3.5: cosmetic code differences
 NOOP = "noop" # 1-1.5: zero output impact, pin by default
...
```
```

2. `src/models/decisions.py`

Decision-related Pydantic models. All models must use:

- ``model_config = ConfigDict(frozen=True)`` for immutability where appropriate
- ``Field(...)`` with descriptions
- Proper validators where needed

```
```python
class Alternative(BaseModel):
 """One option the agent considered at a decision point."""
 value: str
```
```

```
reasoning: str = ""
```

```
class DecisionPoint(BaseModel):
    """A single branching point in the agent's execution."""
    id: str = Field(default_factory=lambda: f"dp-{uuid4().hex[:8]}")
    sequence_number: int
    timestamp: datetime = Field(default_factory=lambda: datetime.now(UTC))
    category: DecisionCategory
    phase: AgentPhase
    question: str
    alternatives: list[Alternative] = Field(min_length=2)
    chosen: str
    reasoning: str
    confidence: float = Field(ge=0.0, le=1.0)
    parent_decision_id: Optional[str] = None
    locked: bool = False
    cost_usd: float = 0.0
    tokens_in: int = 0
    tokens_out: int = 0
    cache_hit: bool = False
```

```
class ExecutionRecord(BaseModel):
    """Record of a code execution attempt in the sandbox."""
    id: str = Field(default_factory=lambda: f"exec-{uuid4().hex[:8]}")
    timestamp: datetime = Field(default_factory=lambda: datetime.now(UTC))
    code_snapshot: str
    command: str
    stdout: str = ""
    stderr: str = ""
    exit_code: int
    duration_ms: int
    artifacts_produced: list[str] = Field(default_factory=list)
```

```
class LLMCallRecord(BaseModel):
    """Record of a single LLM API call."""
    id: str = Field(default_factory=lambda: f"llm-{uuid4().hex[:8]}")
    timestamp: datetime = Field(default_factory=lambda: datetime.now(UTC))
    phase: AgentPhase
    messages: list[dict] # Full messages array sent to LLM
    response: str
    structured_output: Optional[dict] = None
    model: str
    temperature: float
    tokens_in: int
    tokens_out: int
    latency_ms: int
    cost_usd: float
```

```

        decision_point_id: Optional[str] = None
    ...

### 3. src/models/traces.py
The complete trace and run models:
```python
class RunMetadata(BaseModel):
 """Metadata about a single agent run."""
 run_id: str = Field(default_factory=lambda: f"run-{uuid4().hex[:8]}")
 task_description: str
 llm_provider: str # e.g., "openrouter"
 model_routing: dict[str, str] = Field(# Phase → model mapping used in
this run
 default_factory=dict, # e.g., {"planning":
"google/gemini-2.5-flash",
 description="Map of agent phase to model used" # "coding":
"anthropic/claude-sonnet-4"}
)
 temperature: float
 started_at: datetime = Field(default_factory=lambda: datetime.now(UTC))
 completed_at: Optional[datetime] = None
 status: RunStatus = RunStatus.PENDING
 parent_run_id: Optional[str] = None
 fork_point: Optional[str] = None
 lock_config: Optional["PathLockConfig"] = None
 total_llm_calls: int = 0
 total_tokens: int = 0
 total_cost_usd: float = 0.0
 error: Optional[str] = None

class DecisionTrace(BaseModel):
 """Complete trace of a single agent run."""
 metadata: RunMetadata
 decisions: list[DecisionPoint] = Field(default_factory=list)
 executions: list[ExecutionRecord] = Field(default_factory=list)
 llm_calls: list[LLMCallRecord] = Field(default_factory=list)

class PathLockConfig(BaseModel):
 """Configuration for path locking on a replay or fork."""
 source_run_id: str
 locked_decision_ids: Optional[list[str]] = None # Lock specific decisions
 lock_through_sequence: Optional[int] = None # Lock all up to N
 lock_categories: Optional[list[DecisionCategory]] = None
 overrides: dict[str, str] = Field(default_factory=dict) # decision_id -> new
value (for forks)

class CostEstimate(BaseModel):

```

```

 """Predicted cost for a run before execution."""
 model_routing: dict[str, str] # Phase → model mapping being estimated
 estimated_tokens_in: int
 estimated_tokens_out: int
 estimated_cost_usd: float
 confidence_low: float
 confidence_high: float
 breakdown: dict[str, float] = Field(
 default_factory=dict,
 description="Cost breakdown by phase: {'planning': 0.006, 'coding': 0.072,
...}"
)
 cost_by_model: dict[str, float] = Field(
 default_factory=dict,
 description="Cost breakdown by model: {'google/gemini-2.5-flash': 0.006,
...}"
)
 ...

```

### 4. src/models/comparison.py

Models for comparing runs:

```
```python
```

```
class DecisionAlignment(BaseModel):
```

```
    """How a decision aligns between two runs."""
```

```
    category: DecisionCategory
```

```
    question: str
```

```
    run_a_choice: Optional[str] = None
```

```
    run_b_choice: Optional[str] = None
```

```
    match: bool
```

```
class OutcomeDiff(BaseModel):
```

```
    """Outcome differences between two runs."""
```

```
    code_line_count: dict[str, int]          # {run_id: count}
```

```
    libraries_used: dict[str, list[str]]     # {run_id: [libs]}
```

```
    execution_time_ms: dict[str, int]
```

```
    anomalies_found: dict[str, int]
```

```
    total_cost_usd: dict[str, float]
```

```
class RunComparison(BaseModel):
```

```
    """Full comparison between two runs."""
```

```
    run_a_id: str
```

```
    run_b_id: str
```

```
    decision_alignment: list[DecisionAlignment]
```

```
    outcome_diff: OutcomeDiff
```

```
    variance_score: float = Field(ge=0.0, le=1.0)
```

```
...

```

5. src/llm/base.py – LLM Provider Interface

```
```python
```

```
class LLMMessage(BaseModel):
 role: Literal["system", "user", "assistant"]
 content: str
```

```
class LLMResponse(BaseModel):
 content: str
 structured: Optional[dict] = None
 tokens_in: int
 tokens_out: int
 latency_ms: int
 model: str # Actual model used (e.g., "google/gemini-2.5-
flash")
 cost_usd: float
```

```
class LLMProvider(ABC):
 """Abstract base for LLM providers.
```

Supports per-call model routing: the same provider instance can send requests to different models. This enables our multi-model strategy where planning uses Gemini, coding uses Sonnet, and evaluation uses GPT-4o-mini – all through one OpenRouter API key.

```
"""
```

```
@abstractmethod
```

```
async def complete(
 self,
 messages: list[LLMMessage],
 model: Optional[str] = None, # Override model for this
```

```
call
```

```
 response_format: Optional[type[BaseModel]] = None,
 temperature: Optional[float] = None,
 max_tokens: int = 4096,
```

```
) -> LLMResponse:
 """Send messages to the LLM and get a response.
```

```
 Args:
```

```
 messages: Conversation messages to send.
 model: Model to use for this specific call. If None, uses the
 provider's default model. This enables per-phase routing
 (e.g., Gemini for planning, Sonnet for coding).
 response_format: If provided, instruct the model to return JSON
 matching this Pydantic schema.
 temperature: Override temperature for this call.
 max_tokens: Maximum output tokens.
```

```
"""
```

```

 ...

 @abstractmethod
 def estimate_cost(self, model: str, tokens_in: int, tokens_out: int) -> float:
 """Estimate cost in USD for given token counts on a specific model."""
 ...

 @property
 @abstractmethod
 def provider_name(self) -> str: ...
...

6. src/sandbox/base.py – Sandbox Interface
```python
class ExecutionResult(BaseModel):
    stdout: str = ""
    stderr: str = ""
    exit_code: int
    duration_ms: int
    artifacts: list[Path] = Field(default_factory=list)
    timed_out: bool = False

class SandboxProvider(ABC):
    """Abstract base for code execution sandboxes."""

    @abstractmethod
    async def execute(
        self,
        command: str,
        cwd: Path,
        timeout: int = 120,
        env: Optional[dict[str, str]] = None,
    ) -> ExecutionResult: ...

    @abstractmethod
    async def install_packages(
        self, packages: list[str], cwd: Path
    ) -> ExecutionResult: ...

    @abstractmethod
    async def setup_workspace(self, run_id: str) -> Path:
        """Create an isolated workspace directory for a run."""
        ...

    @abstractmethod
    async def cleanup_workspace(self, run_id: str) -> None: ...
...

```

7. src/tracing/base.py – Tracer Interface

```
```python
class DecisionTracer(ABC):
 """Abstract base for decision trace capture."""

 @abstractmethod
 async def record_decision(self, decision: DecisionPoint) -> None: ...

 @abstractmethod
 async def record_execution(self, execution: ExecutionRecord) -> None: ...

 @abstractmethod
 async def record_llm_call(self, call: LLMCallRecord) -> None: ...

 @abstractmethod
 async def get_decisions(self) -> list[DecisionPoint]: ...

 @abstractmethod
 async def finalize(self, status: RunStatus, error: Optional[str] = None) ->
DecisionTrace: ...
...

```

### ### 8. src/locking/base.py – Path Locker Interface

```
```python
class PathLocker(ABC):
    """Abstract base for path locking mechanisms."""

    @abstractmethod
    def is_locked(self, decision_id: str) -> bool: ...

    @abstractmethod
    def get_locked_value(self, decision_id: str) -> Optional[str]: ...

    @abstractmethod
    def get_lock_prompt(self, question: str, locked_value: str) -> str:
        """Generate the prompt injection text for a locked decision."""
        ...

    @abstractmethod
    def verify_decision(self, decision: DecisionPoint) -> bool:
        """Verify that a decision matches the locked value. Returns True if
matches or unlocked."""
        ...
...

```

9. src/agent/base.py – Agent Interface

```

```python
class AgentEventType(str, Enum):
 RUN_STARTED = "run_started"
 PHASE_CHANGED = "phase_changed"
 DECISION_MADE = "decision_made"
 CODE_WRITTEN = "code_written"
 EXECUTION_STARTED = "execution_started"
 EXECUTION_COMPLETED = "execution_completed"
 ERROR_HIT = "error_hit"
 RECOVERY_STARTED = "recovery_started"
 RUN_COMPLETED = "run_completed"
 COST_UPDATE = "cost_update"

class AgentEvent(BaseModel):
 event_type: AgentEventType
 run_id: str
 timestamp: datetime = Field(default_factory=lambda: datetime.now(UTC))
 phase: Optional[AgentPhase] = None
 payload: dict = Field(default_factory=dict)

class EventHandler(Protocol):
 async def handle(self, event: AgentEvent) -> None: ...

class CodingAgent(ABC):
 """Abstract base for the coding agent."""

 @abstractmethod
 async def run(
 self,
 task: str,
 lock_config: Optional[PathLockConfig] = None,
 event_handler: Optional[EventHandler] = None,
) -> DecisionTrace: ...
...

```

### 10. src/db/models.py – SQLAlchemy ORM models

Map the Pydantic models to database tables. Key tables:

- `runs` (id, task\_description, status, metadata JSON, created\_at, completed\_at)
- `decision\_points` (id, run\_id FK, sequence\_number, category, question, chosen, alternatives JSON, reasoning, locked, cost\_usd)
- `execution\_records` (id, run\_id FK, code\_snapshot, command, stdout, stderr, exit\_code, duration\_ms)
- `llm\_calls` (id, run\_id FK, phase, messages JSON, response, tokens\_in, tokens\_out, latency\_ms, cost\_usd)

Use JSON columns for complex nested data (alternatives, messages).



Create an Alembic migration for these tables.

```
11. src/models/__init__.py – Re-export everything
Make imports clean: `from src.models import DecisionPoint, DecisionTrace,
RunMetadata`
```

CRITICAL CODING STANDARDS (apply to ALL files):

- Every class has a docstring explaining its purpose
- Every method has type hints on all params and return type
- Use `from \_\_future\_\_ import annotations` at top of every file
- Field descriptions on all Pydantic model fields
- Validators for any business logic constraints
- No mutable defaults – use Field(default\_factory=...)

## Tests for this phase

Create tests in tests/unit/test\_models.py:

1. test\_decision\_point\_serialization: Create a DecisionPoint, serialize to JSON, deserialize back, assert equality
2. test\_decision\_point\_requires\_min\_alternatives: Assert that < 2 alternatives raises ValidationError
3. test\_confidence\_bounds: Assert confidence outside 0-1 raises ValidationError
4. test\_decision\_trace\_round\_trip: Full trace with decisions, executions, llm\_calls – serialize to JSON and back
5. test\_path\_lock\_config\_modes: Test the three locking modes (by IDs, by sequence, by category) are mutually representable
6. test\_run\_metadata\_defaults: Assert run\_id is auto-generated, status defaults to PENDING
7. test\_cost\_estimate\_structure: Verify the breakdown dict is properly nested

## Manual verification

bash

```
Models import cleanly
docker-compose exec app python -c "from src.models import DecisionPoint, DecisionTra

Alembic migration works
docker-compose exec app alembic upgrade head
docker-compose exec app alembic downgrade -1
docker-compose exec app alembic upgrade head

Tables exist in Postgres
docker-compose exec db psql -U agent -d deterministic_agent -c "\dt"

Tests pass
docker-compose exec app pytest tests/unit/test_models.py -v
```

## Phase 2: LLM Provider + Sandbox + Cache Implementations

### What we're building

Concrete implementations of the three infrastructure interfaces: a unified OpenRouter LLM provider (one class, all models), subprocess-based sandbox, and tiered cache (in-memory + Redis + filesystem). These are the "plumbing" that every agent operation flows through.

### Which evaluation criteria this addresses

- **Architecture & Extensibility (25%):** Single provider class handles all models — adding a new model is changing one config string, not writing a new class. This is a deliberately simpler architecture than the typical "one class per provider" pattern because OpenRouter's unified API makes that abstraction unnecessary.
- **Systems Thinking (20%):** Sandbox timeout handling, Redis connection failure graceful degradation, cache invalidation logic, per-model cost tracking

### Prompt for AI IDE

```
Implement the infrastructure layer for the deterministic agent. Read these
existing
interfaces before starting:
- src/llm/base.py (LLMProvider ABC — note the `model` param on complete())
- src/sandbox/base.py (SandboxProvider ABC)
- src/cache/__init__.py (CacheBackend Protocol)
- src/config.py (Settings, especially LLMConfig with per-phase model routing)

1. src/llm/openrouter_provider.py
```

Implement a SINGLE LLMProvider for OpenRouter. This is the only LLM provider class in the project – OpenRouter gives us access to Claude, GPT, Gemini, DeepSeek, and 300+ other models through one OpenAI-compatible API.

Key design decision: We intentionally do NOT have separate AnthropicProvider, OpenAIProvider, etc. OpenRouter's API is OpenAI-compatible, so we use the openai Python SDK with a custom base\_url. One class handles all models.

```
```python
```

```
from openai import AsyncOpenAI
```

```
# Pricing table: model prefix → (input_cost_per_1M, output_cost_per_1M)
```

```
# OpenRouter passes through provider pricing – these match official rates.
```

```
MODEL_PRICING: dict[str, tuple[float, float]] = {  
    # Gemini (planning – best structured reasoning value)  
    "google/gemini-2.5-flash":      (0.30, 2.50),  
    "google/gemini-2.5-flash-lite": (0.10, 0.40),  
    # Claude (coding + recovery – highest first-pass code success)  
    "anthropic/claude-sonnet-4":    (3.00, 15.00),  
    "anthropic/claude-haiku-4.5":   (1.00, 5.00),  
    # GPT (evaluation – cheapest reliable JSON classifier)  
    "openai/gpt-4o-mini":           (0.15, 0.60),  
    "openai/gpt-4o":                (2.50, 10.00),  
    # DeepSeek (fallback – 95% cheaper than frontier models)  
    "deepseek/deepseek-chat-v3.2":  (0.28, 0.42),  
    "deepseek/deepseek-chat-v3.1":  (0.15, 0.75),  
}
```

```
class OpenRouterProvider(LLMProvider):
```

```
    """
```

```
    Unified LLM provider via OpenRouter.
```

```
    Uses the OpenAI SDK with a custom base_url. Supports per-call model routing:  
    the same instance can call Gemini for planning, Sonnet for coding, and  
    GPT-4o-mini for evaluation – all with one API key.
```

```
    Why one class instead of one-per-provider:
```

- OpenRouter normalizes the API surface – all models speak OpenAI format
 - Eliminates the provider factory pattern entirely
 - Adding a new model = adding one line to MODEL_PRICING
 - The `model` parameter on complete() handles routing, not class hierarchy
- ```
 """
```

```
 def __init__(self, config: LLMConfig):
```

```
 self._config = config
```

```
 self._client = AsyncOpenAI(
 api_key=config.api_key.get_secret_value(),
 base_url=config.base_url,
```

```

 default_headers={
 "HTTP-Referer": "https://github.com/deterministic-agent",
 "X-Title": "Deterministic Agent",
 },
)
 self._default_model = config.model_planning # Fallback if no model
specified

 async def complete(
 self,
 messages: list[LLMMessage],
 model: Optional[str] = None,
 response_format: Optional[type[BaseModel]] = None,
 temperature: Optional[float] = None,
 max_tokens: int = 4096,
) -> LLMResponse:
 """
 Send completion request to any model via OpenRouter.

 The `model` parameter is how per-phase routing works:
 - _plan() passes model=config.model_planning (Gemini)
 - _code() passes model=config.model_coding (Sonnet)
 - _evaluate() passes model=config.model_evaluation (GPT-4o-mini)
 """
 resolved_model = model or self._default_model
 temp = temperature if temperature is not None else
self._config.temperature

 # Build kwargs
 kwargs = {
 "model": resolved_model,
 "messages": [{"role": m.role, "content": m.content} for m in
messages],
 "temperature": temp,
 "max_tokens": max_tokens,
 }

 # Add JSON response format if requested
 # Different models support this differently via OpenRouter:
 # - GPT models: native JSON mode via response_format
 # - Others: instruction in system prompt + parse response
 if response_format:
 kwargs["response_format"] = {"type": "json_object"}

 start = time.perf_counter()

 try:

```

```

 response = await self._client.chat.completions.create(**kwargs)
 except openai.RateLimitError:
 # Retry with exponential backoff (3 attempts)
 for attempt in range(3):
 wait = 2 ** attempt
 logger.warning(f"Rate limited by {resolved_model}, retry in
{wait}s")

 await asyncio.sleep(wait)
 try:
 response = await
self._client.chat.completions.create(**kwargs)
 break
 except openai.RateLimitError:
 if attempt == 2:
 raise
 except openai.APIError as e:
 logger.error(f"API error from {resolved_model}: {e}")
 # If the primary model fails, try the fallback model
 if resolved_model != self._config.model_fallback:
 logger.warning(f"Falling back to {self._config.model_fallback}")
 kwargs["model"] = self._config.model_fallback
 response = await self._client.chat.completions.create(**kwargs)
 else:
 raise

 latency_ms = int((time.perf_counter() - start) * 1000)

 content = response.choices[0].message.content or ""
 usage = response.usage
 tokens_in = usage.prompt_tokens if usage else 0
 tokens_out = usage.completion_tokens if usage else 0
 actual_model = response.model or resolved_model

 cost = self.estimate_cost(actual_model, tokens_in, tokens_out)

 # Parse structured output if requested
 structured = None
 if response_format and content:
 try:
 structured = json.loads(content)
 except json.JSONDecodeError:
 # Try to extract JSON from markdown fences
 structured = self._extract_json(content)

 logger.info(
 "LLM call completed",
 extra={

```

```

 "model": actual_model,
 "tokens_in": tokens_in,
 "tokens_out": tokens_out,
 "latency_ms": latency_ms,
 "cost_usd": f"{cost:.6f}",
 }
)

```

```

return LLMResponse(
 content=content,
 structured=structured,
 tokens_in=tokens_in,
 tokens_out=tokens_out,
 latency_ms=latency_ms,
 model=actual_model,
 cost_usd=cost,
)

```

```

def estimate_cost(self, model: str, tokens_in: int, tokens_out: int) -> float:
 """Estimate cost using the pricing table."""
 # Match model to pricing (handle versioned model strings like
 # "anthropic/claude-sonnet-4:2025-05-14" by stripping version)
 base_model = model.split(":")[0] if ":" in model else model
 rates = MODEL_PRICING.get(base_model)
 if not rates:
 # Unknown model – estimate conservatively at $1/$5 per 1M
 logger.warning(f"No pricing data for model {model}, using estimate")
 rates = (1.00, 5.00)

 input_cost = (tokens_in / 1_000_000) * rates[0]
 output_cost = (tokens_out / 1_000_000) * rates[1]
 return input_cost + output_cost

```

```

@property
def provider_name(self) -> str:
 return "openrouter"

```

```

...

```

### ### 2. src/llm/factory.py

Simplified factory – in practice just creates OpenRouterProvider, but keeps the abstraction for anyone who wants to use a direct provider instead:

```

```python
def create_llm_provider(config: LLMConfig) -> LLMProvider:
    """Create an LLM provider based on configuration.

```

Default and recommended: OpenRouter (one key, all models).

Also supports direct provider SDKs for users who prefer them.

```
"""
```

```
match config.provider:
    case "openrouter":
        return OpenRouterProvider(config)
    case "openai":
        # Direct OpenAI – uses same class, just different base_url
        config_copy = config.model_copy(update={"base_url":
"https://api.openai.com/v1"})
        return OpenRouterProvider(config_copy)
    case "anthropic":
        # For direct Anthropic, we'd need a separate provider class since
        # their API format differs. For MVP, recommend OpenRouter instead.
        raise ValueError(
            "Direct Anthropic SDK not implemented. Use provider=openrouter "
            "with an OpenRouter API key to access Claude models."
        )
    case _:
        raise ValueError(f"Unknown LLM provider: {config.provider}")
...
```

NOTE: The factory for "openai" reuses OpenRouterProvider with a different base_url

–

because OpenRouter IS OpenAI-compatible. This means we have exactly ONE provider implementation in the entire codebase. This is intentional and a good talking point

for the Architecture evaluation criterion.

4. src/sandbox/subprocess_sandbox.py

Implement SandboxProvider using subprocess:

```
- `setup_workspace(run_id)`:
    - Create directory at {RUNS_DIR}/{run_id}/workspace/
    - Create a Python venv inside it: `python -m venv {workspace}/.venv`
    - Return the workspace Path

- `execute(command, cwd, timeout, env)`:
    - Run command via asyncio.create_subprocess_exec
    - Set timeout using asyncio.wait_for – on timeout, kill the process tree,
      return ExecutionResult with timed_out=True
    - Capture stdout + stderr (limit to 50KB each to prevent memory blowup)
    - Set env vars: merge os.environ with provided env, add venv to PATH
    - Log: command, exit_code, duration_ms

- `install_packages(packages, cwd)`:
    - Run: `{venv}/bin/pip install {' '.join(packages)} --quiet`
```

- Parse stderr for errors
 - Return ExecutionResult
-
- ``cleanup_workspace(run_id)``:
 - Remove the workspace directory tree
 - Handle the case where it doesn't exist (log warning, don't error)

IMPORTANT: The sandbox runs INSIDE the Docker container. We're not doing Docker-in-Docker.

The isolation is: separate venv + subprocess with timeout + resource limits.

5. src/cache/memory_cache.py

In-memory LRU cache implementing CacheBackend:

- Use ``OrderedDict`` for LRU behavior
- TTL support: store (value, expires_at) tuples, lazy-expire on get
- Max size in MB: estimate value sizes, evict LRU when exceeded
- Thread-safe: use `asyncio.Lock` (not `threading.Lock`)

6. src/cache/redis_cache.py

Redis cache implementing CacheBackend:

- Use ``redis.asyncio`` client
- All operations have try/except – on `ConnectionError`, log warning and return `None/False`
 - (graceful degradation, never crash the agent because Redis is down)
- Support TTL natively (Redis SETEX)
- Key prefix: "dagent:" to namespace our keys
- Include these specialized methods beyond the Protocol:
 - ``increment_cost(run_id, amount) -> float``: INCRBYFLOAT, atomic
 - ``publish_event(channel, event_json)``: for streaming
 - ``ping()`` -> `bool``: health check

7. src/cache/tiered.py

TieredCache orchestrating L1 (memory) → L2 (Redis) → L3 (filesystem):

- ``get()``: Check L1, then L2, then L3. Backfill higher tiers on hit.
- ``set()``: Always write to L1 + L2. Write to L3 only if `persist=True`.
- ``delete()``: Delete from all tiers.
- Constructor takes optional Redis – if None, L2 falls back to a second `InMemoryCache` instance (for when Redis is unavailable).

8. src/cache/filesystem_cache.py

Filesystem cache implementing CacheBackend:

- Stores JSON files at `{RUNS_DIR}/.cache/{key_hash}.json`
- No TTL (persistent)

- Key hashing: SHA-256 of key, use first 16 chars as filename
- Atomic writes: write to .tmp file, then os.rename

9. Update src/dependencies.py

Wire everything together in the DI container:

- ``startup()`` creates: Settings, LLM provider (OpenRouterProvider), Redis client, TieredCache, DB session factory
- ``shutdown()`` closes: Redis connection, DB engine
- Expose getters: ``get_llm()``, ``get_cache()``, ``get_db()``, ``get_sandbox()``
- If Redis connection fails during startup, log warning and continue without it
- On startup, verify OpenRouter connectivity by calling the provider's health check
(a minimal completion to a free model – or just check the API key is valid)

CODING STANDARDS:

- All network calls have timeouts
- All file operations use context managers
- Retry logic uses tenacity or manual exponential backoff
- Log at appropriate levels: DEBUG for cache hits, INFO for LLM calls, WARNING for fallbacks, ERROR for failures
- No secrets in logs – mask API keys

Tests for this phase

Create tests:

tests/unit/test_llm_provider.py:

1. `test_openrouter_message_formatting`: Mock the AsyncOpenAI client, verify messages are converted to OpenAI format correctly
2. `test_per_call_model_routing`: Call `complete()` with different model strings, verify each request uses the specified model (mock client, check kwargs)
3. `test_cost_calculation_gemini`: Gemini Flash, 4K in 2K out → verify \$0.0062
4. `test_cost_calculation_sonnet`: Claude Sonnet 4, 4K in 4K out → verify \$0.072
5. `test_cost_calculation_gpt4o_mini`: GPT-4o-mini, 2K in 500 out → verify \$0.0006
6. `test_cost_calculation_unknown_model`: Unknown model uses conservative estimate
7. `test_rate_limit_retry`: Mock `RateLimitError` on first call, success on second, verify retry happened
8. `test_fallback_on_api_error`: Mock `APIError` on primary model, verify fallback model is tried
9. `test_json_response_parsing`: Mock response with JSON content, verify structured output is parsed
10. `test_json_extraction_from_markdown_fences`: Response wrapped in ````json````, verify it's extracted correctly

tests/unit/test_sandbox.py:

1. test_execute_simple_command: Run "echo hello", verify stdout
2. test_execute_timeout: Run "sleep 10" with 1s timeout, verify timed_out=True
3. test_workspace_creation_and_cleanup: Create workspace, verify exists, cleanup, verify gone

tests/unit/test_cache.py:

1. test_memory_cache_set_get: Set value, get it back
2. test_memory_cache_ttl_expiry: Set with TTL=1, sleep 2, verify None
3. test_memory_cache_lru_eviction: Fill to max, add one more, verify oldest evicted
4. test_tiered_cache_backfill: Set in L3 only, get from tiered, verify L1 now has it
5. test_redis_graceful_degradation: With no Redis, verify tiered cache still works

Manual verification

bash

```

# LLM provider works with per-phase routing (requires valid OpenRouter key in .env)
docker-compose exec app python -c "
import asyncio
from src.llm.factory import create_llm_provider
from src.config import get_settings
from src.llm.base import LLMMessage

async def test():
    settings = get_settings()
    llm = create_llm_provider(settings.llm)

    # Test planning model (Gemini 2.5 Flash)
    resp = await llm.complete(
        [LLMMessage(role='user', content='Say hello in JSON: {"greeting": "..."}')],
        model=settings.llm.model_planning,
    )
    print(f'[Planning] Model: {resp.model}, Cost: \${resp.cost_usd:.6f}')

    # Test evaluation model (GPT-4o-mini)
    resp = await llm.complete(
        [LLMMessage(role='user', content='Respond with only: {"status": "ok"}')],
        model=settings.llm.model_evaluation,
    )
    print(f'[Evaluation] Model: {resp.model}, Cost: \${resp.cost_usd:.6f}')

    # Test coding model (Claude Sonnet 4)
    resp = await llm.complete(
        [LLMMessage(role='user', content='Write a Python hello world in 2 lines')],
        model=settings.llm.model_coding,
    )
    print(f'[Coding] Model: {resp.model}, Cost: \${resp.cost_usd:.6f}')

asyncio.run(test())
"

# Expected: Three different models used, three different cost levels
# Planning should be cheapest (~$0.001), Evaluation (~$0.0004), Coding most (~$0.01)

# Sandbox works
docker-compose exec app python -c "
import asyncio
from src.sandbox.subprocess_sandbox import SubprocessSandbox

async def test():
    sb = SubprocessSandbox()
    ws = await sb.setup_workspace('test-123')
    result = await sb.execute('python -c "print(2+2)"', cwd=ws)
    print(f'stdout: {result.stdout}, exit: {result.exit_code}')

```

```

        await sb.cleanup_workspace('test-123')
    asyncio.run(test())
    """

# Redis cache works
docker-compose exec app python -c """
import asyncio
from src.cache.redis_cache import RedisCache
async def test():
    cache = RedisCache('redis://redis:6379/0')
    await cache.set('test', 'hello', ttl=60)
    val = await cache.get('test')
    print(f'Got: {val}')
asyncio.run(test())
"""

```

Phase 3: Agent Core (Plan → Code → Execute → Evaluate)

What we're building

The actual coding agent — the state machine that takes a task description, uses the LLM to plan and code a solution, executes it in the sandbox, evaluates the result, and iterates. This is the heart of the system. No decision tracing yet — just the agent loop.

Which evaluation criteria this addresses

- **Problem Decomposition (25%):** The agent's phase decomposition (plan→code→execute→evaluate) IS the problem decomposition
- **Systems Thinking (20%):** Error recovery, iteration limits, timeout handling
- **CORE requirement:** "The Agent"

Prompt for AI IDE

```

Build the coding agent core. Read these files first:
- src/agent/base.py (CodingAgent ABC, AgentEvent)
- src/llm/base.py (LLMProvider, LLMMessage — note `model` param on complete())
- src/llm/openrouter_provider.py (OpenRouterProvider with per-call model routing)
- src/sandbox/base.py (SandboxProvider, ExecutionResult)
- src/models/ (all models)
- src/config.py (AgentConfig, LLMConfig with per-phase model fields)

```

IMPORTANT ARCHITECTURE CONTEXT:

We use a multi-model strategy via OpenRouter. The SAME LLMProvider instance routes to DIFFERENT models per agent phase:

- Planning → config.llm.model_planning (Gemini 2.5 Flash – structured reasoning)
- Coding → config.llm.model_coding (Claude Sonnet 4 – best code generation)
- Evaluation → config.llm.model_evaluation (GPT-4o-mini – cheap JSON classifier)
- Recovery → config.llm.model_recovery (Claude Sonnet 4 – code comprehension)

Each `_call_llm()` invocation passes the phase-appropriate model string. The provider handles routing transparently.

1. src/agent/prompts.py

Define all prompts as string constants or template functions. This file is the SINGLE SOURCE OF TRUTH for every prompt sent to the LLM. Key prompts:

SYSTEM_PROMPT: Sets up the agent's role as a coding agent that must make explicit decisions, enumerate alternatives, and output structured JSON for each decision.

PLANNING_PROMPT: Given a task description, produce a structured plan as JSON:

```
```json
{
 "decisions": [
 {
 "question": "Which stock ticker(s) to analyze?",
 "category": "data_selection",
 "alternatives": [
 {"value": "AAPL", "reasoning": "..."},
 {"value": "TSLA", "reasoning": "..."},
 {"value": "SPY", "reasoning": "..."}
],
 "chosen": "TSLA",
 "reasoning": "...",
 "confidence": 0.75
 }
],
 "implementation_plan": "Step-by-step plan for coding..."
}
```

The prompt MUST instruct the LLM to consider at least 2 alternatives per decision, and to output decisions for: ticker, timeframe, data fields, anomaly method, anomaly parameters, report format, API framework, visualization library.

CODING\_PROMPT: Given the plan and decisions, write complete Python code that:

- Pulls data from yfinance
- Detects anomalies using the chosen method
- Generates a report
- Exposes a FastAPI/Flask endpoint
- Includes proper error handling

- Output as a JSON with keys: "files" (dict of filename -> content), "requirements" (list of pip packages)

EVALUATION\_PROMPT: Given execution results (stdout, stderr, exit\_code), assess:

- Did the code run successfully?
- Did it produce the expected outputs?
- What errors occurred and what recovery strategy to use?

Output as JSON: {"status": "success"|"partial"|"failed", "assessment": "...", "recovery\_strategy": "...", "next\_action": "fix"|"retry"|"abort"}

RECOVERY\_PROMPT: Given the error, the failing code, and the assessment, produce fixed code. Include the decision about what recovery approach to take.

LOCK\_DECISION\_PROMPT: Template function that takes (question, locked\_value) and returns a prompt injection: "For {question}, you MUST choose {locked\_value}. Do not consider alternatives. Set locked=true in your response."

Each prompt should be a function that takes context parameters and returns a formatted string. Use triple-quoted f-strings with clear structure.

### ### 2. src/agent/runner.py – The CodingAgent Implementation

Implement the CodingAgent ABC. This is the main class.

```
```python
```

```
class AgentRunner(CodingAgent):
```

```
    """
```

```
    Coding agent that uses per-phase model routing for cost optimization.
```

```
    Each phase uses the optimal model for its task:
```

- Planning: Gemini 2.5 Flash (structured reasoning, cheap)
- Coding: Claude Sonnet 4 (highest first-pass code quality)
- Evaluation: GPT-4o-mini (cheap classification)
- Recovery: Claude Sonnet 4 (code comprehension)

```
    All models are accessed through a single OpenRouter provider instance.
```

```
    The `model` parameter on _call_llm() handles the routing.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        llm: LLMProvider,
```

```
        sandbox: SandboxProvider,
```

```
        config: AgentConfig,
```

```
        llm_config: LLMConfig,
```

```
        # Needed for per-phase model routing
```

```
    ):
```

```
        self._llm = llm
```

```

self._sandbox = sandbox
self._config = config
self._llm_config = llm_config
self._event_handler: Optional[EventHandler] = None

def _get_model(self, phase: AgentPhase) -> str:
    """Get the configured model for a given agent phase.

    Uses LLMConfig.get_model_for_phase() which checks for override first,
    then returns the phase-specific model.
    """
    return self._llm_config.get_model_for_phase(phase)

async def run(
    self,
    task: str,
    lock_config: Optional[PathLockConfig] = None,
    event_handler: Optional[EventHandler] = None,
) -> DecisionTrace:
    self._event_handler = event_handler
    run_id = f"run-{uuid4().hex[:8]}"
    workspace = await self._sandbox.setup_workspace(run_id)

    # Build the tracer and locker (will be injected in Phase 4/5)
    # For now, use simple containers
    decisions: list[DecisionPoint] = []
    executions: list[ExecutionRecord] = []
    llm_calls: list[LLMCallRecord] = []

    try:
        # Phase 1: PLANNING (uses Gemini 2.5 Flash)
        await self._emit(AgentEvent(event_type=AgentEventType.PHASE_CHANGED,
                                   run_id=run_id,
phase=AgentPhase.PLANNING))
        plan = await self._plan(task, run_id, decisions, llm_calls)

        # Phase 2-4: CODE → EXECUTE → EVALUATE loop
        for iteration in range(self._config.max_iterations):
            # CODING (uses Claude Sonnet 4)
            await
self._emit(AgentEvent(event_type=AgentEventType.PHASE_CHANGED,
                       run_id=run_id,
phase=AgentPhase.CODING))
            code_files, requirements = await self._code(
                task, plan, decisions, executions, llm_calls, run_id
            )

```

```

        # Write files to workspace
        await self._write_files(workspace, code_files)

        # Install dependencies
        if requirements:
            await self._sandbox.install_packages(requirements, workspace)

        # EXECUTING (no LLM call – pure subprocess)
        await
self._emit(AgentEvent(event_type=AgentEventType.PHASE_CHANGED,
                        run_id=run_id,
phase=AgentPhase.EXECUTING))
        exec_result = await self._execute(workspace, code_files, run_id,
executions)

        # EVALUATING (uses GPT-4o-mini)
        await
self._emit(AgentEvent(event_type=AgentEventType.PHASE_CHANGED,
                        run_id=run_id,
phase=AgentPhase.EVALUATING))
        assessment = await self._evaluate(
            exec_result, code_files, task, decisions, llm_calls, run_id
        )

        if assessment["status"] == "success":
            return self._build_trace(run_id, task, decisions, executions,
                                    llm_calls, RunStatus.SUCCESS)

        if assessment["next_action"] == "abort":
            return self._build_trace(run_id, task, decisions, executions,
                                    llm_calls, RunStatus.FAILED,
                                    error=assessment["assessment"])

        # RECOVERY (uses Claude Sonnet 4)
        plan = await self._recover(
            task, plan, assessment, decisions, executions, llm_calls,
run_id
        )

        return self._build_trace(run_id, task, decisions, executions,
llm_calls,
                                RunStatus.FAILED, error="Max iterations
exceeded")

    except Exception as e:
        logger.exception(f"Agent run {run_id} crashed: {e}")
        return self._build_trace(run_id, task, decisions, executions,

```



```
llm_calls,
```

```
RunStatus.PARTIAL, error=str(e))
```

```
finally:
```

```
    # Don't cleanup workspace – keep artifacts for inspection
```

```
    pass
```

```
...
```

Implement each private method. CRITICAL: each method passes the phase-appropriate model to `_call_llm()`:

- `_plan()`: Uses `self._get_model(AgentPhase.PLANNING)` → Gemini 2.5 Flash. Send `PLANNING_PROMPT`, parse response into decisions list + plan text. Each decision from the LLM response becomes a `DecisionPoint` added to the decisions list.
- `_code()`: Uses `self._get_model(AgentPhase.CODING)` → Claude Sonnet 4. Send `CODING_PROMPT` with plan + decisions context. Parse response to extract file contents dict and requirements list. Return both.
- `_write_files()`: Write each file from the code dict to the workspace directory. Create subdirectories as needed. No LLM call.
- `_execute()`: Run the main Python file in the workspace. Detect the entry point (`main.py` or the file with `'if __name__'`). Create `ExecutionRecord`. No LLM call.
- `_evaluate()`: Uses `self._get_model(AgentPhase.EVALUATING)` → GPT-4o-mini. Send `EVALUATION_PROMPT` with execution results. Parse assessment.
- `_recover()`: Uses `self._get_model(AgentPhase.RECOVERING)` → Claude Sonnet 4. Send `RECOVERY_PROMPT`. This may produce new decisions (`error_recovery` category). Update the plan and return for next iteration.
- `_build_trace()`: Construct the complete `DecisionTrace` from accumulated data. Set `metadata.model_routing` to record which models were used per phase:

```
{ "planning": "google/gemini-2.5-flash", "coding": "anthropic/claude-sonnet-4", ... }
```
- `_emit()`: If `event_handler` is set, call it with the event. If not, no-op.
- `_call_llm(messages, phase, response_format=None)`: Central method that all LLM interactions go through. Records `LLMCallRecord`, calculates cost, handles structured output parsing. Resolves model from phase:

```
```python
async def _call_llm(
 self,
 messages: list[LLMMessage],
 phase: AgentPhase,
```

```

 response_format: Optional[type[BaseModel]] = None,
) -> LLMResponse:
 """Central LLM call method with per-phase model routing."""
 model = self._get_model(phase)
 response = await self._llm.complete(
 messages=messages,
 model=model,
 response_format=response_format,
)
 # Record LLM call...
 return response
...

```

### ### 3. src/agent/parsers.py

Response parsing utilities:

- `parse_json_from_llm(text: str) -> dict`: Extract JSON from LLM response that may include markdown code fences or preamble text. Handle: bare JSON, ````json` fenced, mixed text with JSON block. Raise `ParseError` with the raw text if unparseable.
- `parse_planning_response(response: dict) -> tuple[list[DecisionPoint], str]`: Convert the raw planning JSON into a list of `DecisionPoint` models + plan text.
- `parse_coding_response(response: dict) -> tuple[dict[str, str], list[str]]`: Extract files dict and requirements list from coding response.
- `parse_evaluation_response(response: dict) -> dict`: Validate and return the assessment dict.

Make parsers defensive – LLMs don't always follow instructions perfectly. Each parser should handle common deviations (missing fields, extra fields, wrong types) with sensible defaults rather than crashing.

CRITICAL: The agent must actually work end-to-end. After this phase I should be able to run the agent on the stock task and get a working codebase as output. Don't stub anything – implement fully.

## Tests for this phase

- ```
tests/unit/test_parsers.py:
```
1. `test_parse_json_bare`: Parse raw JSON string
 2. `test_parse_json_fenced`: Parse ````json ... ```` block
 3. `test_parse_json_with_preamble`: Parse "Here's the plan:\n```json..."
 4. `test_parse_json_invalid`: Verify `ParseError` raised with raw text preserved

5. test_parse_planning_response: Full planning JSON → DecisionPoint list

tests/integration/test_agent_runner.py:

1. test_agent_run_with_mock_llm: Mock the LLM to return predefined

planning/coding/

evaluation responses. Verify the full loop completes, decisions are captured, files are written to workspace.

2. test_agent_handles_execution_failure: Mock LLM responses where first code fails (exit code 1), recovery produces fixed code. Verify iteration works.

3. test_agent_max_iterations: Mock LLM that never produces passing code. Verify it stops at max_iterations and returns FAILED status.

Manual verification (THE BIG TEST)

bash

```

# Run the agent for real on the stock task – verify per-phase model routing
docker-compose exec app python -c "
import asyncio
from src.agent.runner import AgentRunner
from src.llm.factory import create_llm_provider
from src.sandbox.subprocess_sandbox import SubprocessSandbox
from src.config import get_settings

async def test():
    settings = get_settings()
    llm = create_llm_provider(settings.llm)
    sandbox = SubprocessSandbox()
    agent = AgentRunner(
        llm=llm, sandbox=sandbox,
        config=settings.agent, llm_config=settings.llm,
    )

    trace = await agent.run(
        'Pull stock data from yfinance, detect anomalous trading days '
        'using a statistical method, generate a summary report with '
        'visualizations, and expose results through a simple API endpoint.'
    )

    print(f'Status: {trace.metadata.status}')
    print(f'Model routing: {trace.metadata.model_routing}')
    print(f'Decisions: {len(trace.decisions)}')
    for d in trace.decisions:
        print(f'  {d.category}: {d.question} → {d.chosen}')
    print(f'Cost: \${trace.metadata.total_cost_usd:.4f}')

    # Verify different models were used per phase
    models_used = set()
    for call in trace.llm_calls:
        models_used.add(call.model)
        print(f'  [{call.phase}] model={call.model} cost=\${call.cost_usd:.6f}')
    print(f'Distinct models used: {len(models_used)} – {models_used}')
    # Should see 3 distinct models: Gemini, Sonnet, GPT-4o-mini

asyncio.run(test())
"

# Verify the generated code actually works
ls runs/run-*/workspace/
cat runs/run-*/workspace/main.py

```

```
# Try running the generated code
cd runs/run-*/workspace/ && python main.py
```

Phase 4: Decision Tracing

What we're building

Replace the bare lists in AgentRunner with the proper DecisionTracer implementation. Add incremental persistence (JSONL append), database storage, and the ability to list/view/search traces.

Which evaluation criteria this addresses

- **CORE requirement:** "Decision Trace"
- **Systems Thinking (20%):** Incremental persistence means crash recovery works

Prompt for AI IDE

```
Implement decision tracing. Read:
- src/tracing/base.py (DecisionTracer ABC)
- src/agent/runner.py (current bare list approach)
- src/models/decisions.py, src/models/traces.py
- src/db/models.py (ORM models)
- src/cache/tiered.py (TieredCache)

### 1. src/tracing/tracer.py

Implement DecisionTracer:
```python
class PersistentTracer(DecisionTracer):
 """
 Decision tracer with incremental persistence.

 Writes are append-only to JSONL during the run (crash recovery).
 On finalize(), the complete trace is written to both the database
 and the filesystem as a clean JSON file.
 """
 def __init__(
 self,
 run_id: str,
 task: str,
 runs_dir: Path,
 db_session: Optional[AsyncSession] = None,
 cache: Optional[TieredCache] = None,
 llm_config: Optional[LLMConfig] = None,
```

```
): ...
...
```

Implementation details:

- On init: create {runs\_dir}/{run\_id}/ directory, open {run\_id}/trace.jsonl for append
- `record\_decision()`: Append DecisionPoint as JSON line to trace.jsonl, store in memory list, update cost counters, optionally write to Redis for streaming
- `record\_execution()`: Same pattern for ExecutionRecord
- `record\_llm\_call()`: Same pattern for LLMCallRecord
- `finalize()`:
  1. Build complete DecisionTrace model
  2. Write trace.json (pretty-printed, complete)
  3. Write metadata.json (just RunMetadata)
  4. Write to database (if session available)
  5. Update cache
  6. Return the DecisionTrace
- `get\_decisions()`: Return current in-memory list

### 2. src/tracing/store.py

TraceStore: the read-side for traces.

```
```python
```

```
class TraceStore:
```

```
    """Reads and queries traces from database and filesystem."""
```

```
    async def get_trace(self, run_id: str) -> Optional[DecisionTrace]:
```

```
        """Get a complete trace by run ID. Checks cache, then DB, then  
filesystem."""
```

```
    async def list_runs(  
        self,
```

```
        status: Optional[RunStatus] = None,  
        limit: int = 50,  
        offset: int = 0,
```

```
    ) -> list[RunMetadata]:  
        """List run metadata with optional filtering."""
```

```
        """List run metadata with optional filtering."""
```

```
    async def get_decisions_for_run(self, run_id: str) -> list[DecisionPoint]:
```

```
        """Get just the decisions (lighter than full trace)."""
```

```
    async def delete_run(self, run_id: str) -> bool:
```

```
        """Delete a run's trace, artifacts, and DB records."""
```

```
    async def search_runs(  
        self,
```

```
        ticker: Optional[str] = None,
```

```

        method: Optional[str] = None,
        min_cost: Optional[float] = None,
        max_cost: Optional[float] = None,
    ) -> list[RunMetadata]:
        """Search runs by decision content."""
    ...

```

3. Update src/agent/runner.py

Replace the bare lists with PersistentTracer:

- In ``run()``: create PersistentTracer instance
- In ``_plan()``: after parsing each decision from LLM response, call ``tracer.record_decision(decision)``
- In ``_call_llm()``: call ``tracer.record_llm_call(record)``
- In ``_execute()``: call ``tracer.record_execution(record)``
- In the success/failure paths: call ``tracer.finalize(status, error)``
- Ensure the tracer is finalized even on exception (use try/finally)

4. Cost tracking integration

Add a CostTracker that wraps the tracer's cost accumulation:

- After each LLM call, check accumulated cost against `max_cost_usd`
- If exceeded, finalize trace with PARTIAL status and raise `CostLimitExceededError`
- Include cost in every `AgentEvent(event_type=COST_UPDATE)`

The CostTracker should use Redis INCRBYFLOAT when available for atomic cross-process accuracy, falling back to in-memory counter.

IMPORTANT: The tracer must survive crashes. If the agent process dies mid-run, the JSONL file must contain all decisions up to that point. Test this by verifying JSONL is flushed after every write (don't buffer).

Tests

tests/unit/test_tracer.py:

1. `test_incremental_persistence`: Record 3 decisions, read JSONL file, verify 3 lines of valid JSON
2. `test_finalize_creates_complete_trace`: Record decisions + executions + llm_calls, finalize, verify `trace.json` has everything
3. `test_finalize_writes_metadata`: Verify `metadata.json` exists with correct fields
4. `test_cost_tracking`: Record LLM calls with known costs, verify total matches
5. `test_cost_limit_exceeded`: Set `max_cost` to \$0.001, record an LLM call costing \$0.01, verify `CostLimitExceededError` raised

tests/unit/test_trace_store.py:

1. `test_list_runs_empty`: No runs, returns empty list

2. `test_list_runs_with_data`: Create 3 trace files, verify list returns 3
3. `test_get_trace_from_filesystem`: Write a `trace.json`, verify `get_trace` reads it
4. `test_get_trace_from_cache`: Put trace in cache, verify it's served from cache (not filesystem)

Manual verification

```
bash

# Run agent and verify trace files
docker-compose exec app python -c "
# ... (same as Phase 3 manual test)
"

# Check trace files exist
ls runs/run-*/
# Should see: trace.json, trace.jsonl, metadata.json, workspace/

# Verify JSONL is incrementally written (not empty)
cat runs/run-*/trace.jsonl | python -m json.tool --no-ensure-ascii | head -20

# Verify trace.json is complete
cat runs/run-*/trace.json | python -m json.tool | head -50

# Check Postgres has the data
docker-compose exec db psql -U agent -d deterministic_agent \
  -c "SELECT run_id, status, total_cost_usd FROM runs"
```

Phase 5: Path Locking

What we're building

The mechanism to take a previous run's decision trace and force a new run to follow the same decisions. This is the core design challenge per the problem statement.

Which evaluation criteria this addresses

- **CORE requirement:** "Path Locking" — the central deliverable
- **Problem Decomposition (25%):** How you chose to implement locking reveals your thinking
- **Architecture (25%):** The locking mechanism should be cleanly separated from the agent

Prompt for AI IDE

Implement path locking. This is THE core design challenge. Read:

- src/locking/base.py (PathLocker ABC)
- src/models/traces.py (PathLockConfig, DecisionTrace)
- src/agent/runner.py (AgentRunner)
- src/agent/prompts.py (LOCK_DECISION_PROMPT)

Strategy: Three-layer prompt injection with verification

The locking approach is:

1. PROMPT INJECTION: When a decision is locked, the LLM prompt includes "You MUST choose {value}" – the LLM reasons with the constraint baked in
2. OUTPUT VERIFICATION: After the LLM responds, verify the choice matches the lock.
If not, retry up to 3 times with increasingly forceful prompts.
3. DIVERGENCE DETECTION: If the locked choice is no longer valid (API changed, library removed), detect at execution time and report.

We chose prompt injection over post-hoc filtering because the LLM needs to REASON ABOUT the constraint to produce coherent downstream code.

1. src/locking/locker.py

```
```python
```

```
class PromptInjectionLocker(PathLocker):
```

```
 """
```

```
 Path locker that injects constraints into LLM prompts.
```

```
 Three modes:
```

1. Full lock: all decisions from source trace are locked
2. Partial lock: only specified decision IDs are locked
3. Fork lock: upstream locked, one decision overridden, downstream free

```
 """
```

```
 def __init__(self, source_trace: DecisionTrace, config: PathLockConfig):
```

```
 self._source = source_trace
```

```
 self._config = config
```

```
 self._locked_decisions = self._build_lock_map()
```

```
 self._verification_failures: list[dict] = []
```

```
 def _build_lock_map(self) -> dict[str, DecisionPoint]:
```

```
 """Build a map of question -> locked DecisionPoint based on config."""
```

```
 # If locked_decision_ids is set: lock only those IDs
```

```
 # If lock_through_sequence is set: lock all decisions with
```

```
 # sequence_number <= N
```

```
 # If lock_categories is set: lock all decisions in those categories
```

```
 # Apply overrides: if a decision_id is in overrides dict, use the
```

```

override value instead of the original
If none of these are set: lock ALL decisions (full replay)
...

def is_locked(self, question: str, category: DecisionCategory,
 sequence_number: int) -> bool:
 """Check if a decision at this point should be locked.
 Match by question text similarity, not just ID (because IDs may differ
 across runs). Use normalized question matching."""
 ...

def get_locked_value(self, question: str, category: DecisionCategory) ->
Optional[str]:
 """Get the value to lock to. Returns None if not locked."""
 ...

def get_lock_prompt(self, question: str, locked_value: str) -> str:
 """Generate prompt injection text."""
 return (
 f"\n\n🚫 LOCKED DECISION: For the question \"{question}\", "
 f"you MUST choose \"{locked_value}\". "
 f"Do not consider other alternatives for this decision. "
 f"Set locked=true in your response. "
 f"Adapt all subsequent decisions to be compatible with this choice.\n"
)

def verify_decision(self, decision: DecisionPoint) -> bool:
 """Verify decision matches lock. Returns True if matches or unlocked."""
 if not decision.locked:
 return True # Free decision, always valid

 expected = self.get_locked_value(decision.question, decision.category)
 if expected is None:
 return True # Not in lock map

 # Fuzzy match: normalize both values (lowercase, strip, handle synonyms)
 return self._normalize(decision.chosen) == self._normalize(expected)

def _normalize(self, value: str) -> str:
 """Normalize decision values for matching.
 Handles: case, whitespace, common synonyms
 (e.g., 'IQR' == 'interquartile_range')."""
 ...
...

```

```

2. src/locking/question_matcher.py

```

Question matching logic for cross-run alignment:

```
```python
```

```
class QuestionMatcher:
```

```
    """Matches decision questions across runs for locking alignment.
```

```
    Questions from different runs may be phrased differently but mean the same thing.
```

```
    E.g., "Which ticker to analyze?" vs "What stock ticker should we use?"
```

```
    Strategy:
```

```
    1. Exact match on (category, normalized_question)
```

```
    2. If no exact match, fuzzy match on keywords
```

```
    3. If still no match, fall back to sequence-number alignment
```

```
    """
```

```
    @staticmethod
```

```
    def match(
```

```
        current_question: str,
```

```
        current_category: DecisionCategory,
```

```
        current_sequence: int,
```

```
        source_decisions: list[DecisionPoint],
```

```
) -> Optional[DecisionPoint]:
```

```
    """Find the matching decision from source trace."""
```

```
    ...
```

```
...
```

3. Update src/agent/runner.py for locking

Modify the agent to use the PathLocker:

In ``run()``:

```
```python
```

```
If lock_config provided, load source trace and create locker
```

```
locker = None
```

```
if lock_config:
```

```
 source_trace = await trace_store.get_trace(lock_config.source_run_id)
```

```
 if not source_trace:
```

```
 raise ValueError(f"Source run {lock_config.source_run_id} not found")
```

```
 locker = PromptInjectionLocker(source_trace, lock_config)
```

```
...
```

In ``_plan()``:

```
```python
```

```
# For each decision point the LLM is about to make:
```

```
# 1. Check if locker says it's locked
```

```
# 2. If locked, inject the lock prompt into the LLM messages
```

```
# 3. After LLM responds, verify the decision matches
```

```
# 4. If verification fails, retry with stronger prompt (up to 3 times)
# 5. If all retries fail, raise PathLockViolationError

# The tricky part: during planning, ALL decisions come at once in one LLM call.
# So the lock injection happens in the PLANNING_PROMPT itself:
# "For each of these decisions, some are LOCKED (must use specified value):
# - Ticker: LOCKED → AAPL
# - Timeframe: LOCKED → 2y
# - Anomaly method: FREE (choose freely)
# - API framework: LOCKED → fastapi"
...
```

```
### 4. src/locking/exceptions.py
```

```
```python
```

```
class PathLockViolationError(Exception):
```

```
 """Raised when LLM ignores a locked decision after max retries."""
```

```
 def __init__(self, question: str, expected: str, got: str, attempts: int):
```

```
 self.question = question
```

```
 self.expected = expected
```

```
 self.got = got
```

```
 self.attempts = attempts
```

```
class PathLockDivergenceError(Exception):
```

```
 """Raised when a locked choice is no longer valid at runtime."""
```

```
 def __init__(self, question: str, locked_value: str, reason: str):
```

```
 ...
```

```
...
```

CRITICAL IMPLEMENTATION NOTE: The hardest part is that during planning, the LLM makes multiple decisions in ONE response. The locking prompt must list ALL locked decisions upfront in the planning prompt. Then verification must check EACH decision

in the parsed response against the lock map.

For the coding phase, some locked decisions (like "use FastAPI") manifest as code patterns rather than explicit decision JSON. The verification here is lighter – check that import statements and code structure match the locked choices.

## Tests

```
tests/unit/test_locker.py:
```

1. test\_full\_lock\_all\_decisions\_locked: Create a trace with 5 decisions, lock all, verify is\_locked returns True for each
2. test\_partial\_lock\_by\_ids: Lock only decision IDs ["dp-001", "dp-003"], verify dp-002 is not locked
3. test\_partial\_lock\_by\_sequence: Lock through sequence 3, verify

- decisions 1-3 locked, 4+ not locked
4. test\_fork\_lock\_with\_override: Lock all, override dp-003 with new value, verify dp-003 returns the override value
  5. test\_verify\_decision\_match: Locked decision chosen="AAPL", verify returns True
  6. test\_verify\_decision\_mismatch: Locked decision chosen="AAPL" but decision has chosen="TSLA", verify returns False
  7. test\_normalize\_handles\_synonyms: "IQR" matches "interquartile\_range"
  8. test\_question\_matcher\_exact: Match by exact (category, question)
  9. test\_question\_matcher\_fuzzy: "Which ticker?" matches "What stock ticker should we use?"
  10. test\_question\_matcher\_sequence\_fallback: No text match, falls back to sequence number alignment

tests/integration/test\_path\_locking.py:

1. test\_locked\_replay\_produces\_same\_decisions: Run agent once, lock all decisions, replay – verify all decisions match (mock LLM for speed)
2. test\_partial\_lock\_free\_decisions\_may\_vary: Lock first 3, verify decisions 4+ can differ between runs
3. test\_lock\_violation\_raises: Mock LLM that ignores lock, verify PathLockViolationError after 3 retries

## Manual verification

bash

```
1. Run the agent once, note the run ID
2. Replay with full lock
docker-compose exec app python -c "
import asyncio
from src.agent.runner import AgentRunner
from src.locking.locker import PromptInjectionLocker
from src.models import PathLockConfig
... setup ...

Replay run-abc123 with all decisions locked
trace = await agent.run(
 task='...',
 lock_config=PathLockConfig(source_run_id='run-abc123')
)

Compare decisions
for orig, locked in zip(original_trace.decisions, trace.decisions):
 match = '✓' if orig.chosen == locked.chosen else '✗'
 print(f'{match} {orig.question}: {orig.chosen} → {locked.chosen}')
"
```

---

## Phase 6: CLI

### What we're building

The command-line interface using Click + Rich for pretty output. Commands: run, replay, trace, list, fork, compare, estimate.

### Which evaluation criteria this addresses

- **CORE requirement:** "CLI: run the agent, replay a locked path, view decision traces"

### Prompt for AI IDE

```
Build the CLI using Click and Rich. Read:
- src/agent/runner.py (AgentRunner)
- src/tracing/store.py (TraceStore)
- src/models/ (all models)
- src/config.py (Settings)

src/cli/main.py

Use Click group with subcommands. Use Rich for tables, trees, progress indicators.
```python
import click
from rich.console import Console
from rich.table import Table
from rich.tree import Tree
from rich.progress import Progress
from rich.panel import Panel

console = Console()

@click.group()
def cli():
    """The Deterministic Agent – capture, replay, and explore decision paths."""
    pass

@cli.command()
@click.argument('task', required=False)
@click.option('--task-file', '-f', type=click.Path(exists=True), help='Read task from file')
@click.option('--model', '-m', default=None, help='LLM model override')
@click.option('--max-cost', type=float, default=None, help='Max cost in USD')
@click.option('--max-iterations', type=int, default=None, help='Max agent iterations')
```

```

@click.option('--pin-low', is_flag=True, help='Auto-pin LOW/N00P variance
decisions')
@click.option('--no-confirm', is_flag=True, help='Skip cost confirmation')
@click.option('--verbose', '-v', is_flag=True, help='Stream decisions in real-
time')
def run(task, task_file, model, max_cost, max_iterations, pin_low, no_confirm,
verbose):
    """Run the agent on a task."""
    # 1. Load task from argument or file
    # 2. Show cost estimate with Rich panel
    # 3. Ask for confirmation (unless --no-confirm)
    # 4. Run agent with progress display
    # 5. Show summary table of decisions made
    # 6. Print run ID for future reference

@cli.command()
@click.argument('run_id')
@click.option('--verify', is_flag=True, help='Verify output matches original')
def replay(run_id, verify):
    """Replay a previous run with locked decisions."""
    # 1. Load source trace
    # 2. Show what will be locked
    # 3. Run agent with PathLockConfig(source_run_id=run_id)
    # 4. If --verify, diff the decisions and report mismatches

@cli.command()
@click.argument('run_id')
@click.option('--format', 'fmt', type=click.Choice(['tree', 'json', 'table']),
default='tree')
@click.option('--decisions-only', is_flag=True)
def trace(run_id, fmt, decisions_only):
    """View the decision trace for a run."""
    # tree format: Rich Tree with decisions as branches
    # json format: pretty-printed JSON
    # table format: Rich Table with columns for each decision field

@cli.command(name='list')
@click.option('--status', type=click.Choice(['success', 'failed', 'partial']))
@click.option('--limit', '-n', type=int, default=20)
def list_runs(status, limit):
    """List recorded runs."""
    # Rich Table: run_id, status, decisions count, cost, timestamp

@cli.command()
@click.argument('run_id')
@click.option('--decision', '-d', required=True, help='Decision ID or sequence
number to override')

```

```
@click.option('--choice', '-c', required=True, help='New choice value')
@click.option('--compare', is_flag=True, help='Auto-compare with parent after
completion')
def fork(run_id, decision, choice, compare):
    """Fork from a decision point with a different choice."""

@cli.command()
@click.argument('run_id_1')
@click.argument('run_id_2')
@click.option('--format', 'fmt', type=click.Choice(['table', 'json']),
default='table')
def compare(run_id_1, run_id_2, fmt):
    """Compare two runs side by side."""

@cli.command()
@click.argument('task', required=False)
@click.option('--task-file', '-f', type=click.Path(exists=True))
@click.option('--single-model', '-m', default=None, help='Override all phases to
use one model')
@click.option('--compare-strategies', is_flag=True, help='Compare multi-model vs
single-model cost')
def estimate(task, task_file, single_model, compare_strategies):
    """Estimate cost without running. Shows per-phase model routing and
breakdown."""
    \`\`
```

Make the CLI output beautiful with Rich:

- Use Rich Panel for the cost estimate box – show per-phase model + cost:

Cost Estimate	
Strategy:	Multi-model via OpenRouter
Planning:	google/gemini-2.5-flash \$0.006
Coding:	anthropic/claude-sonnet-4 \$0.072
Evaluation:	openai/gpt-4o-mini \$0.001
Recovery:	anthropic/claude-sonnet-4 \$0.027
Estimated total: \$0.106 (range: \$0.06-\$0.19)	
Budget:	\$2.00

- If `--compare-strategies`, show a second panel with single-model cost:

Using claude-sonnet-4 for ALL phases: \$0.34/run

Using multi-model routing: \$0.10/run (70% savings)

- Use Rich Tree for decision traces (decisions as branches, alternatives as leaves)
- Use Rich Table for run listings – include a "models" column showing distinct count
- Use Rich Progress for long-running operations
- Use color coding: green for locked+matched, yellow for free, red for violations
- Print the run ID prominently at the end so users can reference it

For the real-time verbose mode (--verbose), create a simple EventHandler that prints each event as it happens using Rich:

- Decision made: show question + chosen value + which model made the decision
- Code written: show filenames
- Execution: show pass/fail with timing
- Error: show error message in red
- Model switch: show when a different model is being used for the next phase

Register the CLI entry point in pyproject.toml:

```
[project.scripts]
deterministic-agent = "src.cli.main:cli"
```

Manual verification

```
bash

# All CLI commands work
docker-compose exec app deterministic-agent --help
docker-compose exec app deterministic-agent run --help
docker-compose exec app deterministic-agent list
docker-compose exec app deterministic-agent run "Pull stock data..." --no-confirm -v
docker-compose exec app deterministic-agent trace <run-id> --format tree
docker-compose exec app deterministic-agent replay <run-id>
docker-compose exec app deterministic-agent compare <run-id-1> <run-id-2>
docker-compose exec app deterministic-agent estimate "Pull stock data..." --compare-
```

Phase 7: Record 5 Runs + Tests

What we're building

Execute 5 genuine runs demonstrating path variance. Write the tests that prove trace completeness and locking correctness. This is a deliverable checkpoint — after this phase, all CORE requirements are met.

Which evaluation criteria this addresses

- **CORE requirement:** "At least 5 recorded runs demonstrating genuine path variance"
- **CORE requirement:** "Tests covering: decision trace completeness, path locking correctness"
- **Code Quality & Tests (15%):** Meaningful tests, not just coverage padding

Prompt for AI IDE

We need to record 5 genuine agent runs and write comprehensive tests.

1. scripts/record_runs.py

A script that runs the agent 5 times on the SAME task description, with NO locking.

Each run should produce a genuinely different solution.

```
```python
```

```
TASK = """
```

```
Pull stock data from yfinance (you choose which ticker(s), timeframe, and indicators),
```

```
detect anomalous trading days using a statistical method of your choosing, generate a summary report with visualizations, and expose the results through a simple API endpoint.
```

```
"""
```

```
async def record_five_runs():
```

```
 settings = get_settings()
```

```
 llm = create_llm_provider(settings.llm)
```

```
 sandbox = SubprocessSandbox()
```

```
 agent = AgentRunner(
```

```
 llm=llm, sandbox=sandbox,
```

```
 config=settings.agent, llm_config=settings.llm,
```

```
)
```

```
for i in range(5):
```

```
 print(f"\n{'='*60}")
```

```
 print(f"RUN {i+1}/5")
```

```
 print(f"{'='*60}\n")
```

```
 trace = await agent.run(TASK)
```

```
 # Print summary
```

```
 print(f"Run ID: {trace.metadata.run_id}")
```

```
 print(f"Status: {trace.metadata.status}")
```

```
 print(f"Models used: {trace.metadata.model_routing}")
```

```

print(f"Decisions:")
for d in trace.decisions:
 print(f" [{d.category}] {d.question} → {d.chosen}")
print(f"Cost: ${trace.metadata.total_cost_usd:.4f}")

Per-model cost breakdown
model_costs: dict[str, float] = {}
for call in trace.llm_calls:
 model_costs[call.model] = model_costs.get(call.model, 0) +
call.cost_usd
for model, cost in sorted(model_costs.items()):
 print(f" {model}: ${cost:.6f}")
print()
...

```

After running, produce a variance report:

- For each decision category, show which choices were made across all 5 runs
- Highlight decisions where all 5 runs chose the same thing (low variance)
- Highlight decisions where all 5 runs chose differently (high variance)
- This validates our variance classification from the SDD

### 2. tests/unit/test\_decision\_trace\_completeness.py

```
```python
```

```
"""Tests that decision traces are complete and well-formed."""
```

```
class TestDecisionTraceCompleteness:
```

```

    def test_every_run_has_minimum_decisions(self, recorded_runs):
        """Each run must have at least 5 decision points covering the core
choices."""
        for trace in recorded_runs:
            assert len(trace.decisions) >= 5

    def test_required_decision_categories_present(self, recorded_runs):
        """Every run must have decisions in: data_selection, algorithm_selection,
architecture."""
        required = {DecisionCategory.DATA_SELECTION,
DecisionCategory.ALGORITHM_SELECTION,
                    DecisionCategory.ARCHITECTURE}
        for trace in recorded_runs:
            categories = {d.category for d in trace.decisions}
            assert required.issubset(categories)

    def test_decisions_have_alternatives(self, recorded_runs):
        """Every decision must have at least 2 alternatives."""
        for trace in recorded_runs:
            for decision in trace.decisions:

```

```

        assert len(decision.alternatives) >= 2

def test_decisions_have_reasoning(self, recorded_runs):
    """Every decision must have non-empty reasoning."""
    for trace in recorded_runs:
        for decision in trace.decisions:
            assert decision.reasoning.strip()

def test_sequence_numbers_are_ordered(self, recorded_runs):
    """Sequence numbers must be monotonically increasing."""
    for trace in recorded_runs:
        seq_nums = [d.sequence_number for d in trace.decisions]
        assert seq_nums == sorted(seq_nums)

def test_execution_records_present(self, recorded_runs):
    """Each run must have at least one execution record."""
    for trace in recorded_runs:
        assert len(trace.executions) >= 1

def test_llm_calls_recorded(self, recorded_runs):
    """Each run must have at least 2 LLM calls (plan + code)."""
    for trace in recorded_runs:
        assert len(trace.llm_calls) >= 2

def test_multi_model_routing_used(self, recorded_runs):
    """Each run must use multiple distinct models (per-phase routing)."""
    for trace in recorded_runs:
        models_used = {call.model for call in trace.llm_calls}
        assert len(models_used) >= 2, (
            f"Expected multiple models via per-phase routing, "
            f"got only: {models_used}"
        )

def test_model_routing_metadata_populated(self, recorded_runs):
    """Run metadata must record which model was used per phase."""
    for trace in recorded_runs:
        routing = trace.metadata.model_routing
        assert "planning" in routing or "coding" in routing, (
            f"model_routing not populated: {routing}"
        )

def test_evaluation_uses_cheap_model(self, recorded_runs):
    """Evaluation phase should use the cheapest model (GPT-4o-mini)."""
    for trace in recorded_runs:
        eval_calls = [c for c in trace.llm_calls
                       if c.phase == AgentPhase.EVALUATING]
        for call in eval_calls:

```

```

        # Should NOT be using expensive models for evaluation
        assert "sonnet" not in call.model.lower(), (
            f"Evaluation using expensive model: {call.model}"
        )

def test_cost_is_positive(self, recorded_runs):
    """Each run must have a positive total cost."""
    for trace in recorded_runs:
        assert trace.metadata.total_cost_usd > 0

def test_serialization_round_trip(self, recorded_runs):
    """Each trace must survive JSON serialization and deserialization."""
    for trace in recorded_runs:
        json_str = trace.model_dump_json()
        restored = DecisionTrace.model_validate_json(json_str)
        assert restored == trace
...

### 3. tests/unit/test_path_locking_correctness.py
```python
"""Tests that path locking produces correct behavior."""

class TestPathLockingCorrectness:

 def test_full_lock_all_decisions_match(self, recorded_run, mock_llm):
 """Replaying a run with full lock produces identical decisions."""
 # Mock LLM to follow lock instructions
 lock_config = PathLockConfig(source_run_id=recorded_run.metadata.run_id)
 replayed = await agent.run(task, lock_config=lock_config)

 for orig, replay in zip(recorded_run.decisions, replayed.decisions):
 if orig.category not in (DecisionCategory.ERROR_RECOVERY,):
 assert replay.chosen == orig.chosen
 assert replay.locked == True

 def test_partial_lock_respects_boundaries(self):
 """Locking decisions 1-3 doesn't constrain decisions 4+."""
 lock_config = PathLockConfig(
 source_run_id="run-xxx",
 lock_through_sequence=3,
)
 # Decisions 1-3: must match source
 # Decisions 4+: free to vary

 def test_lock_verification_detects_mismatch(self):
 """When LLM ignores a lock, verification catches it."""

```

```

def test_fork_overrides_single_decision(self):
 """Forking changes one decision while locking others."""
 lock_config = PathLockConfig(
 source_run_id="run-xxx",
 overrides={"dp-003": "z_score"}, # Override anomaly method
)

def test_lock_prompt_is_injected(self, mock_llm):
 """Verify the LLM prompt contains lock instructions."""
 # Capture the messages sent to mock LLM
 # Assert that locked decisions appear in the prompt text
...

4. tests/conftest.py additions
```python
@pytest.fixture(scope="session")
def recorded_runs():
    """Load the 5 recorded runs from the runs/ directory."""
    store = TraceStore(runs_dir=Path("runs"))
    runs = asyncio.run(store.list_runs(status=RunStatus.SUCCESS, limit=5))
    traces = [asyncio.run(store.get_trace(r.run_id)) for r in runs]
    assert len(traces) >= 5, "Need at least 5 recorded runs. Run
scripts/record_runs.py first."
    return traces

@pytest.fixture
def recorded_run(recorded_runs):
    """Single recorded run for locking tests."""
    return recorded_runs[0]
...

```

5. Variance analysis output

After running all 5 runs, produce a file: runs/variance_report.json

```

```json
{
 "total_runs": 5,
 "llm_strategy": "multi-model via OpenRouter",
 "model_routing": {
 "planning": "google/gemini-2.5-flash",
 "coding": "anthropic/claude-sonnet-4",
 "evaluation": "openai/gpt-4o-mini",
 "recovery": "anthropic/claude-sonnet-4"
 },
 "unique_tickers": ["AAPL", "TSLA", "MSFT", "SPY", "GOOGL"],
 "unique_methods": ["z_score", "iqr", "isolation_forest"],
 "unique_frameworks": ["fastapi"],

```

```

"decision_variance": {
 "ticker_selection": {"unique_values": 5, "entropy": 2.32},
 "anomaly_method": {"unique_values": 3, "entropy": 1.58},
 "api_framework": {"unique_values": 1, "entropy": 0.0}
},
"cost_summary": {
 "total_all_runs_usd": 0.52,
 "avg_per_run_usd": 0.104,
 "cost_by_model": {
 "google/gemini-2.5-flash": 0.032,
 "anthropic/claude-sonnet-4": 0.41,
 "openai/gpt-4o-mini": 0.004
 },
 "cost_by_phase": {
 "planning": 0.032,
 "coding": 0.36,
 "evaluation": 0.004,
 "recovery": 0.05
 }
},
"runs": [
 {
 "run_id": "run-abc", "ticker": "TSLA", "method": "iqr",
 "cost_usd": 0.10, "models_used": 3, "iterations": 1
 }
]
}
...

```

IMPORTANT: These must be REAL runs with REAL yfinance data and REAL LLM calls. Not mocked. The traces must be committed to the repo as evidence. The variance report MUST include the cost\_by\_model breakdown – this demonstrates the multi-model optimization strategy and is a strong talking point for Systems Thinking evaluation.

## Manual verification

```
bash
```

```
Record 5 runs
docker-compose exec app python scripts/record_runs.py

Verify 5 runs exist
docker-compose exec app deterministic-agent list

Verify variance (at least 3 different tickers and 2 different methods across runs)
cat runs/variance_report.json | python -m json.tool

Run all tests
docker-compose exec app pytest tests/ -v --tb=short

Replay one run and verify decisions match
docker-compose exec app deterministic-agent replay <first-run-id> --verify
```

**After this phase, all CORE requirements are satisfied. Everything below is STRETCH.**

---

## Phase 8: Fork & Explore (STRETCH)

### What we're building

The ability to select any decision point from a previous run, override it with a different choice, lock everything upstream, and let everything downstream run freely. Plus side-by-side comparison.

### Which evaluation criteria this addresses

- **STRETCH: Fork & Explore (10%)**
- **Architecture (25%):** Fork is a natural extension of locking — if the architecture is right, this is small

### Prompt for AI IDE

```
Implement fork-and-explore. Read:
- src/locking/locker.py (PromptInjectionLocker, PathLockConfig)
- src/models/comparison.py (RunComparison)
- src/tracing/store.py (TraceStore)
- src/agent/runner.py (AgentRunner)

1. src/forking/engine.py
```python
class ForkEngine:
    """
    Orchestrates fork-and-explore operations.
```


A fork takes a source run, locks all decisions upstream of the fork point, overrides the fork-point decision, and lets downstream decisions run freely.

```
"""
```

```
def __init__(self, agent: CodingAgent, trace_store: TraceStore):
    self._agent = agent
    self._store = trace_store
```

```
async def fork(
    self,
    source_run_id: str,
    decision_id: str,
    new_choice: str,
    event_handler: Optional[EventHandler] = None,
) -> tuple[DecisionTrace, RunComparison]:
```

```
    """
```

```
    Fork a run at a specific decision point.
```

```
    Returns the new trace AND a comparison with the source run.
```

```
    """
```

```
    source = await self._store.get_trace(source_run_id)
```

```
    # Find the fork point
```

```
    fork_decision = next(
        (d for d in source.decisions if d.id == decision_id
         or str(d.sequence_number) == decision_id),
        None
    )
```

```
    if not fork_decision:
```

```
        raise ValueError(f"Decision {decision_id} not found in run {source_run_id}")
```

```
    # Build lock config: lock upstream, override fork point, free downstream
```

```
    lock_config = PathLockConfig(
        source_run_id=source_run_id,
        lock_through_sequence=fork_decision.sequence_number - 1,
        overrides={fork_decision.id: new_choice},
    )
```

```
    # Run the agent with this lock config
```

```
    forked_trace = await self._agent.run(
        task=source.metadata.task_description,
        lock_config=lock_config,
        event_handler=event_handler,
    )
```

```

        # Set fork metadata
        forked_trace.metadata.parent_run_id = source_run_id
        forked_trace.metadata.fork_point = decision_id

        # Generate comparison
        comparison = self._compare(source, forked_trace)

        return forked_trace, comparison
    ...

### 2. src/analysis/comparator.py
```python
class RunComparator:
 """Compares two runs to identify decision-level differences."""

 def compare(self, run_a: DecisionTrace, run_b: DecisionTrace) ->
RunComparison:
 """
 Align decisions between two runs and compute differences.

 Uses QuestionMatcher to align decisions that may have different IDs
 but answer the same question.
 """
 alignments = []
 for dec_a in run_a.decisions:
 matched = QuestionMatcher.match(
 dec_a.question, dec_a.category, dec_a.sequence_number,
 run_b.decisions
)
 alignments.append(DecisionAlignment(
 category=dec_a.category,
 question=dec_a.question,
 run_a_choice=dec_a.chosen,
 run_b_choice=matched.chosen if matched else None,
 match=matched.chosen == dec_a.chosen if matched else False,
))

 # Add decisions in B that have no match in A
 # ...

 outcome_diff = self._compute_outcome_diff(run_a, run_b)
 variance_score = 1.0 - (sum(a.match for a in alignments) /
len(alignments))

 return RunComparison(
 run_a_id=run_a.metadata.run_id,
 run_b_id=run_b.metadata.run_id,

```

```

 decision_alignment=alignments,
 outcome_diff=outcome_diff,
 variance_score=variance_score,
)
 ...

```

### ### 3. Update CLI

Add the fork and compare commands with Rich output.

- `fork` command: show which decisions are locked/overridden/free before running
- `compare` command: Rich table showing decisions side by side with color coding (green = match, red = different)

Tests:

1. test\_fork\_preserves\_upstream: Fork at decision 3, verify decisions 1-2 match
2. test\_fork\_overrides\_target: Verify the forked decision uses the new value
3. test\_fork\_frees\_downstream: Verify decisions after fork point can differ
4. test\_comparison\_alignment: Two traces with known differences, verify alignment
5. test\_comparison\_variance\_score: All same = 0.0, all different = 1.0

## Phase 9: Path Analysis + Visualization (STRETCH)

### What we're building

Cross-run analysis: decision tree visualization, variance identification, outcome correlation.

### Prompt for AI IDE

Implement cross-run path analysis. Read:

- src/tracing/store.py
- src/analysis/comparator.py
- src/models/

### 1. src/analysis/path\_analyzer.py

```
```python
```

```
class PathAnalyzer:
```

```
    """Analyzes decision paths across multiple runs."""
```

```
    async def analyze(self, run_ids: Optional[list[str]] = None) -> PathAnalysis:
        """
```

```
        Analyze all (or specified) runs. Returns:
```

- Decision tree: merged tree of all paths taken
 - Variance by decision: which decisions vary most
 - Outcome correlation: which choices correlate with better results
- ```
 """
```

```

 def build_decision_tree(self, traces: list[DecisionTrace]) ->
DecisionTreeNode:
 """
 Build a merged decision tree from multiple traces.
 Root → first decision → branches for each unique choice →
 next decision → branches → ...
 """

 def compute_variance_by_decision(
 self, traces: list[DecisionTrace]
) -> list[DecisionVariance]:
 """
 For each decision question, compute:
 - unique_choices: set of values chosen across runs
 - entropy: Shannon entropy (higher = more variance)
 - most_common: mode value
 """

 def correlate_with_outcomes(
 self, traces: list[DecisionTrace]
) -> list[OutcomeCorrelation]:
 """
 For each decision, correlate choices with outcome metrics:
 - execution_success_rate by choice
 - total_cost by choice
 - code_size by choice
 """

class PathAnalysis(BaseModel):
 total_runs: int
 decision_tree: dict # JSON-serializable tree structure
 variance_by_decision: list[DecisionVariance]
 outcome_correlations: list[OutcomeCorrelation]

class DecisionVariance(BaseModel):
 question: str
 category: DecisionCategory
 unique_choices: list[str]
 entropy: float
 most_common: str
 samples: int
 ...

2. CLI: `deterministic-agent analyze`

```

Output a Rich tree showing the merged decision paths:

Decision tree across 5 runs:

```
├── Ticker selection
│ ├── AAPL (2 runs) → [run-abc, run-def]
│ ├── TSLA (2 runs) → [run-ghi, run-jkl]
│ └── SPY (1 run) → [run-mno]
├── Timeframe
│ ├── 2y (3 runs)
│ └── 1y (2 runs)
├── Anomaly method ⚡ HIGHEST VARIANCE
│ ├── z_score (1 run)
│ ├── iqr (2 runs)
│ └── isolation_forest (2 runs)
```

...

Also show a variance ranking table:

| Decision | Unique Choices | Entropy | Highest Variance? |
|----------|----------------|---------|-------------------|
|----------|----------------|---------|-------------------|

## Phase 10: FastAPI Endpoints (STRETCH)

### Prompt for AI IDE

Build FastAPI endpoints exposing all functionality. Read:

- src/agent/runner.py
- src/tracing/store.py
- src/forking/engine.py
- src/analysis/path\_analyzer.py

### src/api/routes.py

Endpoints:

|        |                              |                                                        |
|--------|------------------------------|--------------------------------------------------------|
| POST   | /api/v1/runs                 | - Start a new run (background task)                    |
| GET    | /api/v1/runs                 | - List runs (pagination: limit, offset, status filter) |
| GET    | /api/v1/runs/{run_id}        | - Get run metadata                                     |
| GET    | /api/v1/runs/{run_id}/trace  | - Get full decision trace                              |
| DELETE | /api/v1/runs/{run_id}        | - Delete run                                           |
| POST   | /api/v1/runs/{run_id}/replay | - Replay with locked path                              |
| POST   | /api/v1/runs/{run_id}/fork   | - Fork from decision point                             |

```
Body: {"decision_id": "dp-003", "choice": "z_score"}
```

```
POST /api/v1/compare - Compare two runs
```

```
Body: {"run_a_id": "...", "run_b_id": "..."}


```

```
GET /api/v1/analysis - Cross-run path analysis


```

```
POST /api/v1/estimate - Cost estimate
```

```
Body: {"task": "...", "model": "..."}


```

Key implementation details:

- Long-running operations (run, replay, fork) use FastAPI BackgroundTasks and return immediately with a run\_id + status polling URL
- All responses use Pydantic response models (not raw dicts)
- Error responses follow RFC 7807 (Problem Details)
- Include OpenAPI tags for grouping
- CORS middleware for frontend dev
- Health endpoint: GET /health

```
src/api/app.py


```

Create the FastAPI app:

- Lifespan handler for startup/shutdown (init DI container)
- Exception handlers for PathLockViolationError, CostLimitExceededError
- Request ID middleware (X-Request-ID header)

```
src/api/schemas.py


```

Request/response schemas (separate from domain models):

- RunCreateRequest, RunCreateResponse
- ReplayRequest, ForkRequest
- CompareRequest, CompareResponse
- TraceResponse (the full trace, but with Pydantic serialization)
- ErrorResponse (RFC 7807)

Add Swagger UI customization with description of the system.

---

## Phase 11: Real-Time Streaming (STRETCH)

### Prompt for AI IDE

Add real-time streaming of agent execution via WebSocket + Redis pub/sub.

```
1. src/streaming/publisher.py


```



```

 case AgentEventType.CODE_WRITTEN:
 console.print(f" 📄 Files: {'', ' '.join(event.payload['files'])}")
 case AgentEventType.EXECUTION_COMPLETED:
 status = "✅" if event.payload['exit_code'] == 0 else "❌"
 console.print(f" {status} Execution: exit={
{event.payload['exit_code']}}")
 case AgentEventType.COST_UPDATE:
 console.print(f" 💰 Cost: ${event.payload['total_usd']:.4f}")
 ...

```

### ### 4. Wire into AgentRunner

Pass the event handler through to the agent:

- CLI --verbose creates RichEventHandler
- FastAPI run endpoint creates RedisEventPublisher
- Both can be composed: MultiEventHandler([rich, redis])

## Phase 12: README + Final Polish

### What we're building

The README is 5% of the grade but it frames everything. It should demonstrate clear thinking, honest tradeoffs, and AI usage documentation.

### Prompt for AI IDE

Write a comprehensive README.md. It must include these sections (per evaluation criteria):

1. **Overview** – 3 sentences explaining what this system does and why
2. **Quick Start**
  - Prerequisites: Docker, Docker Compose, OpenRouter API key (one key for all models)
  - `cp .env.example .env` → add your OpenRouter API key (get one at openrouter.ai)
  - `docker-compose up -d`
  - `deterministic-agent run "Pull stock data..."`
  - `deterministic-agent list`
  - `deterministic-agent trace <run-id>`
  - Note: First run costs ~\$0.10 using our multi-model strategy.  
All 5 recorded runs cost ~\$0.50 total.
3. **Architecture** – link to the SDD, include the layer diagram



#### 4. **\*\*LLM Strategy: Multi-Model Routing via OpenRouter\*\*** (THIS IS A DIFFERENTIATOR)

- Explain why we use OpenRouter: one API key, one bill, 300+ models, OpenAI-SDK compatible, switch models by changing a string
- Explain the per-phase routing strategy with rationale for each model choice:
  - Planning → Gemini 2.5 Flash: Native JSON schema enforcement eliminates parsing retries. Built-in thinking mode for better alternative enumeration. \$0.30/\$2.50/1M – 10x cheaper than Sonnet for equivalent structured

reasoning.

- Coding → Claude Sonnet 4: Highest first-pass success rate on multi-file Python.

A model that gets code right on pass 1 saves 3+ retry cycles × 3K tokens each.

The \$3/\$15 unit cost is cheaper than \$0.28/\$0.42 × 4 retries.

- Evaluation → GPT-4o-mini: This is a classification task ("did exit\_code = 0?"), not a reasoning task. GPT-4o-mini's JSON mode is the most reliable at \$0.15/\$0.60 – paying more here is waste.
- Recovery → Claude Sonnet 4: Misdiagnosed errors cascade into wasted cycles. Code comprehension quality justifies the cost.
- Show the cost comparison table: single-model (\$0.34/run) vs multi-model (\$0.10/run)
- Explain how this maps to the .env configuration

#### 5. **\*\*Path Locking Strategy\*\*** (REQUIRED by problem statement)

- Three-layer approach: prompt injection → verification → divergence detection
- Why prompt injection over post-hoc filtering (the LLM must reason with the constraint for coherent downstream code)
- How partial locking and forking extend the base mechanism
- Known limitations: LLM may ignore weak prompts, cross-model replay has phrasing differences (mitigated by fuzzy question matching)

#### 6. **\*\*Decision Point Structure\*\*** (REQUIRED)

- Show the DecisionPoint Pydantic model
- Explain the category taxonomy
- Explain the variance tier classification (HIGH/MEDIUM/LOW/NOOP) and how NOOP decisions can be auto-pinned to save tokens
- Show a real example from a recorded run

#### 7. **\*\*Prompts Used\*\*** (REQUIRED)

- Document every prompt template from src/agent/prompts.py
- For each: what it does, which model it targets, why it's structured that way
- Note: planning prompts are optimized for Gemini's thinking mode, coding prompts for Sonnet's code style, evaluation prompts for GPT-4o-mini's JSON mode

#### 8. **\*\*Variance Analysis\*\***

- Summary of the 5 runs: which decisions varied, which didn't

- The variance tier classification and why it matters for cost optimization
- Include the cost\_by\_model breakdown from variance\_report.json showing that 78% of cost is in coding (where quality matters), <1% in evaluation

#### 9. **Tradeoffs** (REQUIRED)

- Prompt injection vs post-hoc filtering vs constrained decoding
- Multi-model routing vs single model: complexity vs 70% cost reduction
- OpenRouter dependency vs direct provider SDKs: convenience vs latency
- Semantic-level decisions vs fine-grained vs coarse-grained
- Filesystem vs database for trace storage
- Subprocess vs Docker for sandbox
- Temperature 0 is NOT how we achieve reproducibility

#### 10. **AI Tool Usage** (REQUIRED by evaluation criteria)

- What prompts were given to AI tools
- What had to be refactored or fixed
- What design decisions were made by the human
- Specifically: the multi-model strategy, variance tier classification, and path locking approach were human-designed; implementation was AI-assisted

#### 11. **What I'd Do With More Time**

- Checkpoint caching for 90% token savings on replays
- Adaptive model routing: start with cheap model, escalate on failure (DeepSeek → Gemini → Sonnet cascade)
- Cross-provider replay with semantic normalization
- Visual decision tree UI (React frontend)
- A/B testing framework built on fork-and-explore

Include a section on how to run tests: ``docker-compose exec app pytest tests/ -v``

## Verification Checklist — All Requirements

### CORE Requirements

| Requirement                               | Phase   | How to Verify                                                                                         |
|-------------------------------------------|---------|-------------------------------------------------------------------------------------------------------|
| The Agent<br>(plan→code→execute→evaluate) | Phase 3 | <code>deterministic-agent run "..."</code> produces working code                                      |
| Multi-model routing active                | Phase 3 | <code>deterministic-agent trace &lt;id&gt; --format json</code> shows 3+ distinct models in llm_calls |
| Decision Trace (Pydantic, not free text)  | Phase 4 | <code>deterministic-agent trace &lt;id&gt; --format json</code> shows structured data                 |

| Requirement                                      | Phase    | How to Verify                                                                         |
|--------------------------------------------------|----------|---------------------------------------------------------------------------------------|
| Path Locking (replay follows same decisions)     | Phase 5  | <code>deterministic-agent replay &lt;id&gt; --verify</code> shows all decisions match |
| CLI (run, replay, trace)                         | Phase 6  | All CLI commands work                                                                 |
| 5 recorded runs with variance                    | Phase 7  | <code>deterministic-agent list</code> shows 5 successful runs                         |
| Tests (trace completeness, locking correctness)  | Phase 7  | <code>pytest tests/ -v</code> all green                                               |
| README (strategy, structure, prompts, tradeoffs) | Phase 12 | Review README sections                                                                |

### STRETCH Requirements

| Requirement         | Phase    | How to Verify                                                                   |
|---------------------|----------|---------------------------------------------------------------------------------|
| Fork & Explore      | Phase 8  | <code>deterministic-agent fork &lt;id&gt; -d dp-003 -c z_score --compare</code> |
| Path Analysis       | Phase 9  | <code>deterministic-agent analyze</code> shows decision tree + variance         |
| FastAPI endpoints   | Phase 10 | <code>curl http://localhost:8000/api/v1/runs</code> returns JSON                |
| Real-time streaming | Phase 11 | WebSocket connects and receives events during a run                             |

### Evaluation Criteria Mapping

| Dimension (Weight)                 | Where It's Demonstrated                                                                                                                                             |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem Decomposition (25%)        | Phase 1 models, Phase 3 agent phases, variance tiers, per-phase model selection rationale                                                                           |
| Architecture & Extensibility (25%) | Protocols/ABCs, DI container, single OpenRouterProvider with per-call routing (adding model = 1 line), fork=extension of lock                                       |
| Systems Thinking (20%)             | Redis graceful degradation, cost controls, multi-model routing for 70% cost reduction, crash recovery via JSONL, timeout handling, auto-fallback on provider outage |

| Dimension (Weight)            | Where It's Demonstrated                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|
| Code Quality & Tests<br>(15%) | Type hints everywhere, Pydantic validation, meaningful tests in Phase 7 incl. multi-model routing validation |
| Stretch & Real-Time<br>(10%)  | Phases 8-11                                                                                                  |
| README & Reasoning<br>(5%)    | Phase 12 — LLM strategy section with cost analysis is a differentiator                                       |